

POST AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY 1



FINAL REPORT

BUSINESS INTELLIGENCE

E-COMMERCE DATA ANALYSIS

Teacher

Kim Ngoc Bach

Class

E22HTTT

Group

11

Hanoi, 2026

ACKNOWLEDGMENTS

First and foremost, we would like to express our deepest and most sincere gratitude to our instructor, Kim Ngoc Bach, for their dedicated guidance, invaluable feedback, and continuous support throughout the duration of this Business Intelligence course. Their profound expertise, patience, and constant encouragement were instrumental in helping us navigate the rigorous complexities of data engineering, ETL processes, and multidimensional modeling. The transition from theoretical concepts to a practical, high-performance data pipeline would not have been possible without their insightful mentorship.

We also extend our profound thanks to the Faculty of Information Technology 1 at the Post and Telecommunications Institute of Technology for fostering an excellent academic environment. The comprehensive curriculum and access to the necessary resources greatly facilitated our research and practical implementation. We are truly grateful for the solid educational foundation the faculty has provided, which continuously prepares us to tackle real-world challenges in the dynamic IT industry.

Furthermore, we would like to take a moment to thank our families and friends. Their unwavering moral support, understanding, and encouragement during the long hours of research, coding, and system optimization provided us with the mental motivation needed to see this project through to the very end.

Finally, we would like to acknowledge the relentless collective effort and dedication of all team members from Group 11, Class E22HTTT. Building this robust data pipeline and interactive reporting system required overcoming numerous technical hurdles, debugging complex data structures, and continuously optimizing system performance. The successful completion of this project is a true testament to our strong collaboration, late-night problem-solving, and shared passion for data analytics. We are incredibly proud of what we have achieved together.

CONTENT

ACKNOWLEDGMENTS.....	2
LIST OF IMAGES.....	5
INTRODUCTION.....	6
CHAPTER 1: PROJECT OVERVIEW AND TECHNOLOGIES USED.....	7
1.1. Project Objectives & Scope.....	7
1.1.1. Objectives and Significance.....	7
1.1.2. Scope of the Problem.....	7
1.1.3. Expected Outputs.....	8
1.2. Technologies and Tools Used.....	8
1.2.1. Storage Systems and Data Warehouse (Database & Data Warehouse).....	8
1.2.2. Data Processing and Orchestration Platform (Data Processing & Orchestration).....	10
1.2.3. Data Visualization and Business Intelligence (BI) Layer.....	18
1.2.4. System Performance Monitoring Tools (System Monitoring).....	19
1.3. Overview of Business Intelligence and Data Warehouse.....	19
1.3.1. Overall Architecture of a BI System.....	20
1.3.2. Importance of Dimensional Modeling.....	20
1.4. Overall Architecture of the Data Pipeline.....	22
CHAPTER 2: DATA MODEL ANALYSIS AND DESIGN.....	25
2.1. Input Data Profiling.....	25
2.1.1. Dataset Description.....	25
2.1.2. Identifying Raw Data Issues.....	28
2.2. Data Warehouse Design.....	29
2.2.1. Business Process Identification.....	29
2.2.2. Staging Layer Design (ODS - Operational Data Store).....	30
2.2.3. Fact Table Design.....	35
2.2.4. Dimension Table Design.....	38
2.2.5. Schema Selection.....	42
CHAPTER 3: ETL PROCESS IMPLEMENTATION AND DATA CLEANSING.	43
3.1. Data Extraction (Extract).....	43
3.2. Data Transformation and Cleansing (Transform).....	43
3.2.1. Handling Errors and Missing Values.....	44
3.2.2. Data Standardization.....	44
3.2.3. Restructuring and Derived Metrics.....	45
3.3. Loading Data into the Data Warehouse (Load).....	46
CHAPTER 4: DASHBOARD CONSTRUCTION AND INSIGHTS ANALYSIS...	

4.1. Dashboard Design Architecture.....	48
4.1.1. Structure and User Interface (UI).....	48
4.1.2. Metric Calculations and Measure Generation.....	49
4.2. Revenue and Business Performance Analysis (Tab 1).....	49
4.2.1. Strategic Overview Indicators (KPI Cards).....	50
4.2.2. Time-Series Insights and Revenue Trends.....	51
4.3. Geographical Business Analysis (Tab 2).....	52
4.3.1. Spatial Visualization Components.....	52
4.4. Product and Material Performance (Tab 3).....	53
4.4.1. Product Portfolio Visualizations.....	54
4.5. Web Traffic and User Behavior Funnel (Tab 4).....	56
4.5.1. Traffic Landscape.....	56
4.5.2. Conversion Performance & Behavioral Funnels.....	57
4.6. BI System Evaluation.....	57
4.6.1. Architectural Strengths.....	57
4.6.2. Technical Limitations and Future Scope.....	58
CONCLUSION.....	59
REFERENCES.....	60

LIST OF IMAGES

Figure 1.1. Image of Apache Spark.....	10
Figure 1.2. Apache Spark Ecosystem.....	11
Figure 1.3. Components of Apache Spark.....	12
Figure 1.4. Apache Spark Architecture.....	13
Figure 1.5. Apache Airflow Workflow Management Interface.....	14
Figure 1.6. Overall Architecture of Apache Airflow.....	16
Figure 1.7. Image of Metabase.....	18
Figure 1.8. Image of Grafana.....	19
Figure 1.9. Overall Data Pipeline Architecture.....	22
Figure 2.1. Sample data.....	26
Figure 2.2. Fact Constellation Schema (Galaxy Schema).....	42
Figure 3.1. Detailed ETL Data Flow from Staging to Star Schema.....	46
Figure 4.1. Revenue and Business Performance Dashboard (Tab 1).....	50
Figure 4.2. Geographical Business Analysis Dashboard (Tab 2).....	52
Figure 4.3. Product and Material Performance Dashboard (Tab 3).....	54
Figure 4.4. Web Traffic and User Behavior Funnel Dashboard (Tab 4).....	56

INTRODUCTION

In the fiercely competitive modern digital economy, e-commerce platforms generate massive volumes of transactional and behavioral data every single day. However, raw data alone holds little value unless it is properly processed, structured, and analyzed. Business Intelligence (BI) serves as the critical bridge that transforms this discrete transaction data into dynamic reports and actionable insights, enabling organizations to transition from intuition-based strategies to Data-Driven Decision Making.

This project focuses on building a comprehensive Business Intelligence analytical process tailored for an e-commerce system. The core objective is to design a highly scalable and fault-tolerant Data Pipeline capable of handling massive datasets (over 41 million records) with low latency. By leveraging an advanced technology ecosystem, including MongoDB for raw data landing, Apache Spark and Airflow for ETL orchestration, ClickHouse for column-oriented data warehousing, and Metabase for visualization, the project constructs a rigorous multidimensional data model. The ultimate output is an interactive dashboard system designed to solve three central business problems: identifying top-selling products, tracking monthly revenue trends, and decoding customer purchasing behavior.

This report thoroughly documents the entire lifecycle of the project, from raw data extraction to final insight generation. The content is structured into four main chapters:

- **CHAPTER 1: PROJECT OVERVIEW AND TECHNOLOGIES USED** introduces the project's objectives, scope, expected outputs, and details the advanced data technology ecosystem utilized to build the pipeline.
- **CHAPTER 2: DATA MODEL ANALYSIS AND DESIGN** focuses on input data profiling, raw data issue identification, and the comprehensive design of the Data Warehouse, including the Staging Layer (ODS) and a highly optimized Fact Constellation Schema (Galaxy Schema).
- **CHAPTER 3: ETL PROCESS IMPLEMENTATION AND DATA CLEANSING** explains the rigorous data extraction, 12-step transformation, and loading processes using Apache Spark to ensure data integrity and accuracy before ingestion into ClickHouse.
- **CHAPTER 4: DASHBOARD CONSTRUCTION AND INSIGHTS ANALYSIS** presents the final visualizations, derives strategic business insights regarding products, revenue, and customer behavior, and provides a critical evaluation of the BI system.

CHAPTER 1: PROJECT OVERVIEW AND TECHNOLOGIES USED

This chapter provides a foundational overview of the Business Intelligence project. It begins by outlining the core objectives, significance, and scope of the e-commerce data analysis problem. Subsequently, it details the comprehensive technology ecosystem selected to construct the Data Pipeline, including MongoDB, Apache Spark, Apache Airflow, ClickHouse, Metabase, and Grafana. Finally, it presents the overall architectural blueprint that integrates these tools to ensure a highly scalable, decoupled, and automated data flow.

1.1. Project Objectives & Scope

1.1.1. Objectives and Significance

- **Objectives:** The project aims to build a comprehensive Business Intelligence (BI) analytical process, ranging from data extraction and cleaning (ETL), building a dimensional data model (Dimensional Modeling), to data visualization. The core objective is to transform a massive volume of discrete transaction data from an e-commerce system into dynamic reports (Dashboards), providing profound insights into business performance.
- **Practical Significance for Businesses:** In the fiercely competitive e-commerce landscape, effectively exploiting transaction data is key for management to make Data-Driven Decisions. The system helps businesses accurately identify core products, grasp revenue trends, and deeply understand customer behavior. Consequently, businesses can optimize marketing strategies, manage the supply chain, and enhance the user experience.
- **Academic Significance:** The project serves as an opportunity to directly apply theories regarding Data Warehousing and BI to a real-world problem. Through this, skills in database design, data processing, and business analysis thinking are reinforced.

1.1.2. Scope of the Problem

- **In terms of data:** The project focuses on exploiting an e-commerce transaction dataset (Online Retail/E-commerce Dataset). The input data centers on the core attributes of the sales process, such as invoice information, product details, customer identification, transaction time, and geographic region.
- **In terms of business operations:** The scope is limited to analyzing static transaction data, focusing on recorded and completed orders.
- **In terms of system architecture:** The focus of the problem is designing and implementing a multidimensional data model (Star Schema or Snowflake

Schema). This architecture will include a central event table (Fact table: Orders) containing measurement metrics, surrounded by dimension tables (Customer, Product, Time, Region) to provide context for the analysis.

1.1.3. Expected Outputs

The final product of the project is a visual Dashboard system capable of flexible interaction through filters (Slicers/Filters) to comprehensively resolve 3 central analytical problems:

- **Analysis of Top Selling Products:** Provides reports ranking the products and categories that generate the highest revenue and sales volume. This aspect helps businesses evaluate the performance of each item, thereby formulating appropriate inventory management and pricing strategies.
- **Monthly Revenue Analysis:** Visually displays the trend of revenue fluctuations and order volume over time periods (day, month, year). This helps detect seasonality in the business, identifying peak months for resource preparation.
- **Customer Purchasing Behavior Analysis:** Sketches a comprehensive picture of the customer base through multidimensional perspectives such as the geographical region with the highest purchasing power, customer segments, and average order value. The analytical results act as a premise for building targeted marketing campaigns and improving Customer Retention.

1.2. Technologies and Tools Used

To meet the requirements of processing massive transaction data volumes with low latency while ensuring the automation and integrity of the Data Pipeline, the project has selected and implemented an advanced data technology ecosystem. Based on their roles and functions, these technologies are stratified into three core groups as follows:

1.2.1. Storage Systems and Data Warehouse (Database & Data Warehouse)

In the architecture of a modern Business Intelligence (BI) system, a clear separation between the landing zone for raw data and the storage for processed data is a mandatory element to optimize performance. This subsystem includes two core components:

1.2.1.1. MongoDB (NoSQL)

- **General Description:** MongoDB is an open-source non-relational database management system (NoSQL) that operates based on a document-oriented storage mechanism. Instead of using rigid table and row structures like SQL, MongoDB stores data in BSON format (an extension of JSON).

- **Role in the System Architecture:** In this Data Pipeline, MongoDB acts as the "Landing Zone" or Source Database. It is responsible for receiving and storing in its original form the entire volume of raw data extracted directly from the e-commerce system before it is processed.
- **Advantages and Reasons for Selection:**
 - **High Flexibility (Schema-less):** E-commerce data is often inconsistent and constantly changing (for example, a shopping cart can contain many or few products, event logs have different attributes depending on user behavior). MongoDB allows storing these diverse data structures without needing to define a schema in advance.
 - **Outstanding Write Performance:** MongoDB is optimized to process millions of event log records or transaction histories generated continuously with extremely low latency, without affecting the performance of the sales system.
 - **Horizontal Scalability:** Through a sharding mechanism, MongoDB can distribute raw data across multiple server nodes, helping the system easily accommodate massive amounts of data over time.

1.2.1.2. ClickHouse (Column-oriented DBMS)

- **General Description:** ClickHouse is an open-source column-oriented database management system (Column-oriented DBMS), specifically designed for online analytical processing (OLAP) tasks at superior speeds.
- **Role in the System Architecture:** ClickHouse takes on the crucial role as the central Data Warehouse of the entire BI system. Data after the ETL (extract, transform, load) process will be loaded here and structured according to Dimensional Modeling with a Star Schema.
- **Advantages and Reasons for Selection:**
 - **Blazing Fast Query Speed:** Thanks to the column-oriented storage architecture, ClickHouse only reads the columns actually needed for the query instead of scanning the entire row. This minimizes disk I/O, increasing query speed by hundreds of times compared to traditional RDBMS (like MySQL or PostgreSQL) when processing millions, even billions of records.
 - **Excellent Data Compression Capability:** Columnar storage allows ClickHouse to apply extremely efficient data compression algorithms, saving significant storage space and enhancing processing bandwidth on RAM.

- SQL Compatibility and BI Ecosystem: Supports friendly SQL language, easily performing JOINS and Aggregations, while providing smooth integration capabilities with most modern BI platforms.

1.2.2. Data Processing and Orchestration Platform (Data Processing & Orchestration)

1.2.2.1. Apache Spark

a. Introduction to Apache Spark



Figure 1.1. Image of Apache Spark

Apache Spark is an open-source distributed processing system used for large data workloads. Spark enables real-time data processing, receiving data from various sources while simultaneously executing processing on the newly received data (Spark Streaming).

Advantages

Apache Spark brings several benefits, especially concerning high processing speed, support for complex analysis, and ease of use.

- **High Speed and Performance:** A primary advantage of Apache Spark is its ability to process with high speed and performance, especially when compared to Hadoop MapReduce. This stems from the ability to store data in RAM memory instead of continuously writing and reading data to and from disks.
- **Multi-Language Support:** Although written in Scala, it is extended to support other languages such as Java, Python, and R. This makes Spark more flexible and easier to learn and practice.
- **Advanced Analytics Capability:** It combines with Spark SQL to process structured data, MLlib for machine learning, GraphX for graph processing, and Spark Streaming. This allows users to perform complex tasks.

- **Diverse Ecosystem:** It is compatible with multiple systems and integrates with various different platforms.

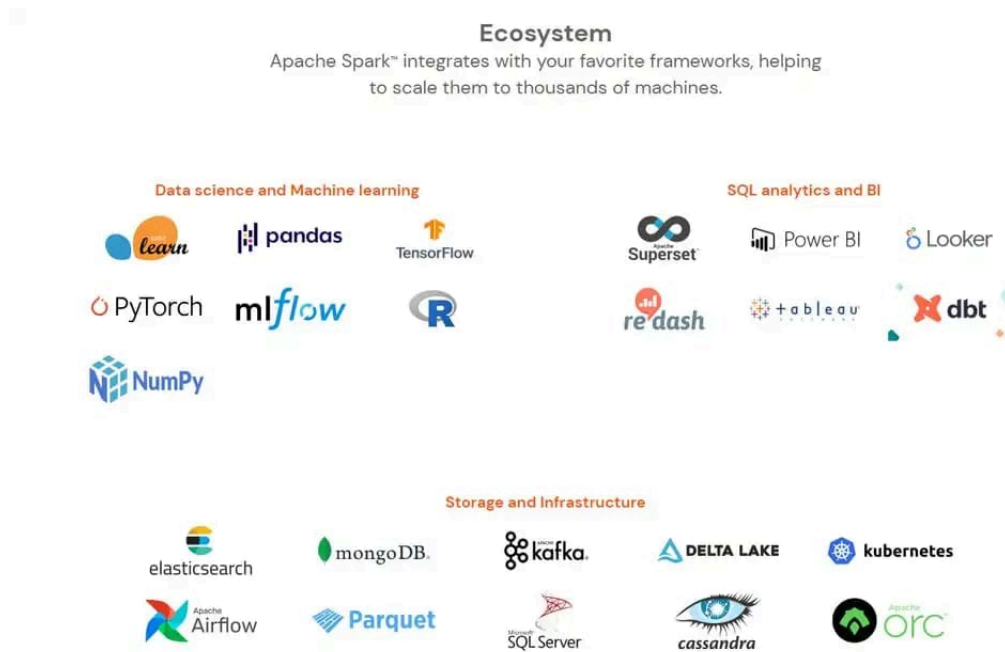


Figure 1.2. Apache Spark Ecosystem

Disadvantages

Although Spark's capabilities are impressive, there are still limitations.

- **Memory Consumption:** Spark leverages a lot of RAM for in-memory calculations, providing the ability to process data at high speeds. This characteristic leads to high memory consumption, resulting in significant operational costs.
- **Limited True Real-time Processing:** In reality, Apache Spark does not have true real-time processing capabilities. Because intrinsically, Spark Streaming relies on micro-batches, processing small groups collected over a predetermined time interval.

b. Components of Apache Spark

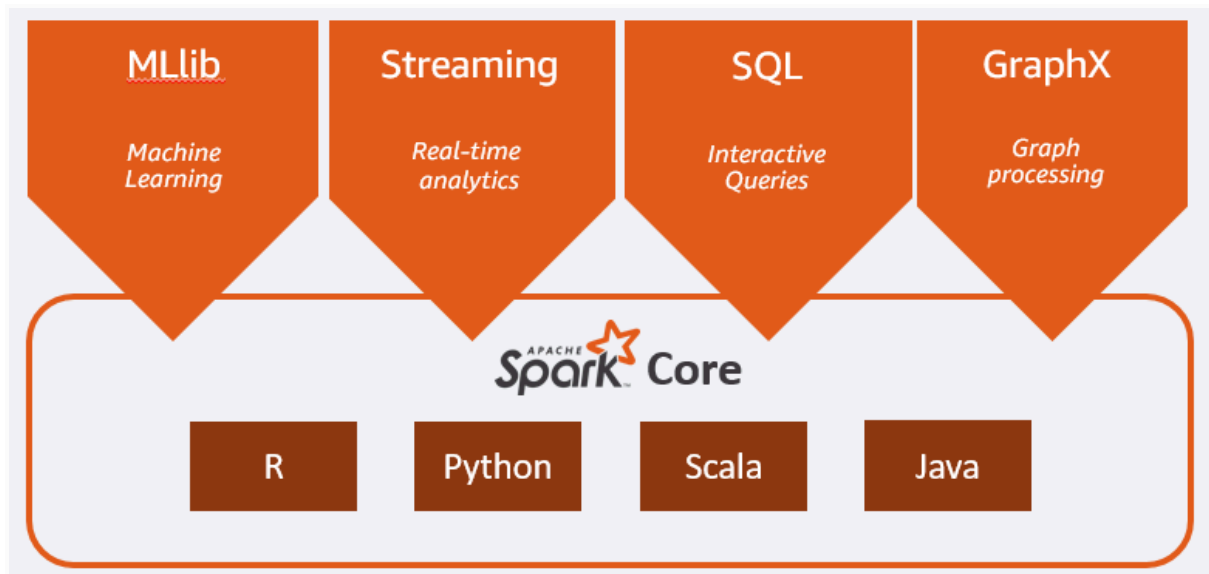


Figure 1.3. Components of Apache Spark

Spark Core: Serves as the foundation for the platform, performing memory management, fault recovery, scheduling, distributing, and monitoring tasks; interacting with storage systems. It provides APIs for Java, Scala, Python, R, hiding the complexity of distributed processing via high-level operators.

MLlib: Is a distributed machine learning platform.

Spark Streaming: Used to process continuous data streams.

Spark SQL: Provides SchemaRDD to support both structured and semi-structured data types.

GraphX: A distributed framework for ETL, analysis, and iterative graph computation. Provides flexible APIs and a suite of graph algorithms, supporting the construction and transformation of large-scale graph structures.

c. Apache Spark Architecture

1. RDD and DAG

In essence, the architecture of Apache Spark is designed for in-memory distributed computing, helping to process large data with incredible speed. Two main concepts in Spark's architecture are Resilient Distributed Dataset (RDD) and Directed Acyclic Graph (DAG).

Resilient Distributed Dataset (RDD)

RDD is an abstract data structure in Spark, representing a collection of immutable objects, partitioned and distributed across nodes in a cluster. RDDs are designed for parallel processing and ensuring fault tolerance.

- **Resilient:** Data can recover from errors through a data recreation mechanism. By re-applying transformations from the original data.
- **Distributed:** RDD is divided into partitions, each partition is stored and processed on a different node in the cluster. Thanks to this, Spark can perform tasks in parallel.
- **Dataset:** An RDD can contain any type of data and can be created from data stored in the local file system.

Directed Acyclic Graph (DAG)

DAG is a directed graph without cycles, used by Spark to represent the sequence of transformations applied to RDDs.

2. Master-Slave Architecture

Apache Spark's architecture is based on the master-slave model, wherein there is a master node coordinating work, and multiple slave nodes executing tasks. This model ensures efficient resource management and can execute in parallel on a computer cluster.

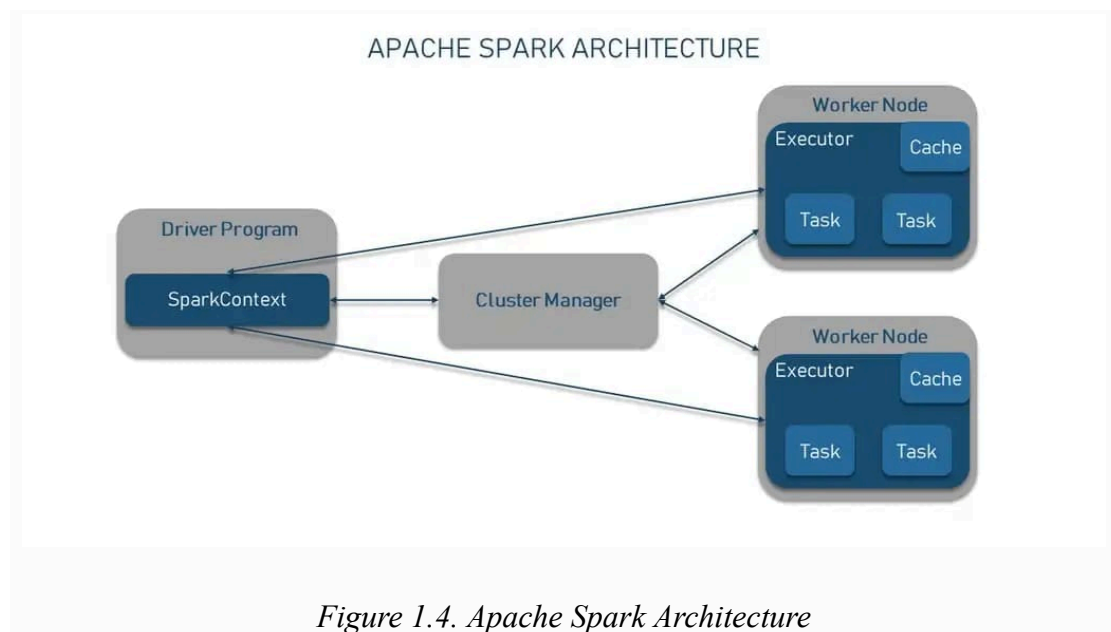


Figure 1.4. Apache Spark Architecture

- **Driver Program:** Is the master node, responsible for running Spark's main function and creating the SparkContext, acting as the entry point for all Spark functions. SparkContext manages communication with the Cluster Manager, partitions work into tasks, and distributes these tasks to the worker nodes.
- **Cluster Manager:** Is the intermediary component, responsible for allocating resources within the cluster.

- **Worker Node:** Are the slave nodes in the cluster, responsible for executing tasks assigned by the Driver.

Operational Mechanism:

- The Driver Program initializes the Spark application, subsequently creating the SparkContext. And builds a DAG based on RDD transformations.
- After that, the SparkContext communicates with the Cluster Manager. It requests CPU and memory resources to allocate to the Worker Nodes.
- After resource allocation, the Cluster Manager initializes Executors on the Worker Nodes. And provides resource information to the Driver.
- Finally, the Driver sends tasks to the Executors via the Task Scheduler. The Executors execute tasks on RDD partitions, then return the results to the Driver.

1.2.2.2. Apache Airflow

a. Introduction to Apache Airflow

Apache Airflow is an open-source platform used to program, schedule, and monitor workflows. Initially developed by Airbnb, Airflow allows users to define data pipelines as programming code (Configuration as Code) using the Python language, making the automation and management of ETL tasks intuitive and easy to control.



Figure 1.5. Apache Airflow Workflow Management Interface

Advantages

Using Apache Airflow brings many outstanding benefits in managing complex data pipelines:

- **Process Coding (Configuration as Code):** Allows data engineers to define pipelines entirely in Python. This facilitates version control, code reuse, and dynamic pipeline generation.
- **Powerful User Interface (UI):** Provides an intuitive Web UI, allowing administrators to easily track the status of workflows, check error logs, and restart failed tasks with just a few operations.
- **Ecosystem and High Integration:** Airflow possesses hundreds of built-in providers, making it easy to connect and coordinate various different systems such as Apache Spark, Kafka, MongoDB, PostgreSQL, or cloud services.
- **Scalability:** The system can scale from a single server to a massive distributed cluster through execution tools like Celery Executor or Kubernetes Executor.

Disadvantages

- **Not a Data Processing System:** Airflow itself does not process data (unlike Spark). It only plays the role of a "conductor" coordinating other tools to work. Abusing Airflow to transport large data between tasks will cause overload.
- **Not Optimized for Real-time Data Streams (Streaming):** Airflow is designed primarily for batch processing workflows or running periodically (cron job). It does not react sensitively to events (event-driven) at the millisecond level like Kafka.
- **Operational Complexity:** Requires significant time to configure, deploy, and maintain an Airflow environment, especially when setting up on a distributed cluster.

b. Overview of Architecture and Main Components

Apache Airflow's architecture is divided into several independent components that communicate closely with each other to ensure smooth workflows.

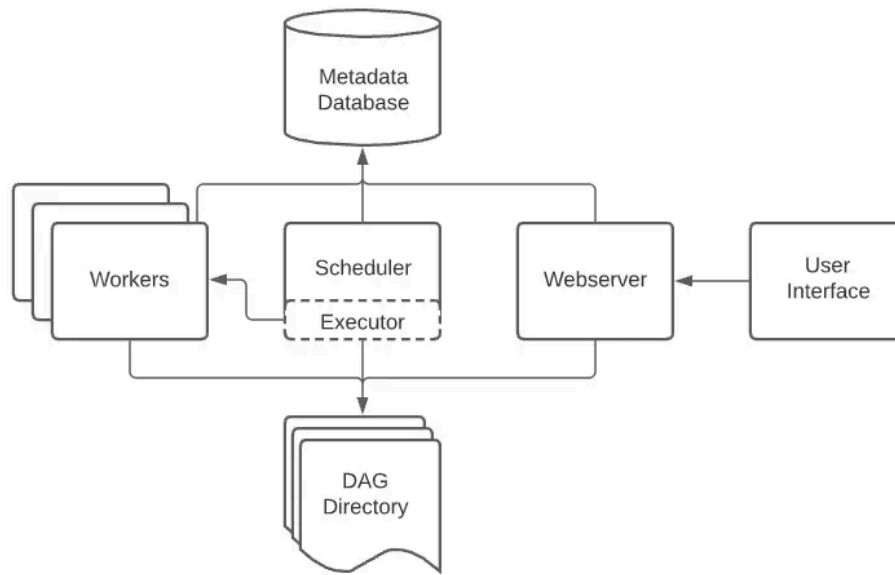


Figure 1.6. Overall Architecture of Apache Airflow

Core Terminology:

- **DAG (Directed Acyclic Graph):** Is the most important concept in Airflow. A DAG represents a workflow, where nodes are tasks and edges specify dependencies between them. The "acyclic" graph ensures workflows are not stuck in infinite loops.
- **Task:** A specific node in a DAG, representing a single unit of work (for example: running a Spark script, or connecting to MongoDB).
- **Operator:** Is a template defining what a Task will do. For example: BashOperator to run terminal commands, PythonOperator to run Python functions, SparkSubmitOperator to trigger a Spark job.

Main Architectural Components:

- **Web Server:** A web application written in Flask, providing a user interface to visualize DAGs, monitor status, view logs, and administer the system.
- **Scheduler:** Is the "brain" of Airflow. It continuously parses DAG files, checks scheduling times and dependency conditions. When the appropriate time arrives, the Scheduler places tasks in a queue for execution.
- **Executor:** The mechanism defining how and where tasks will be run. The Executor interacts with the Scheduler to allocate work to Workers (e.g., LocalExecutor runs on one machine, CeleryExecutor distributes across multiple machines).
- **Worker:** Servers or processes responsible for directly executing tasks assigned by the Scheduler.

- **Metadata Database:** A relational database (usually PostgreSQL or MySQL) storing the entire state, configuration, connection information, and running history of DAGs as well as Tasks.

c. Method of Operation

The operational process of a data pipeline (Pipeline) in Airflow occurs in the following sequential steps:

1. Writing Code and Parsing:

- Users define DAGs and Tasks in Python files.
- The Scheduler component periodically scans the directory containing source code, parses syntax to understand the graph structure, and stores base information in the Metadata Database.

2. Scheduling:

- The Scheduler continuously checks the database to find Tasks that are due to run or have satisfied prerequisite conditions (e.g., Task A finishes before Task B can run).
- When conditions are met, the Task transitions to the Queued state.

3. Allocation and Execution:

- The Scheduler pushes the Task into the queue. The Executor component will recognize and assign this Task to an available Worker.
- The Worker executes the work (e.g., Sends a Spark job call to process data taken from Kafka). The Task's state at this point is Running.

4. Status Update and Monitoring:

- After the Worker finishes running, it reports the result (Success or Failed) back to the Metadata Database.
- The Scheduler receives the result. If successful, it proceeds to trigger the next Task in the DAG. If failed, it can automatically Retry depending on the configuration.
- Administrators can monitor this entire process in real-time on the Web Server interface.

1.2.3. Data Visualization and Business Intelligence (BI) Layer



Figure 1.7. Image of Metabase

Metabase (BI and Visualization Tool):

- **General Description:** Metabase is an open-source Business Intelligence platform, distinguished by a minimalist design, fast response speed, and an extremely friendly user interface (UI). This tool allows users from Data Analysts to Business Users to easily explore data through drag-and-drop actions or writing direct SQL queries.
- **Role in the System Architecture:** Metabase plays the role of the Presentation Layer, providing the final interaction touchpoint for users and management. This tool is configured to connect directly to the central Data Warehouse (ClickHouse). Thanks to ClickHouse's blazing query speed, Metabase can retrieve and display cleaned and modeled data volumes almost instantaneously.
- **Advantages and Applications in the Project:** The combination of Metabase and ClickHouse creates a robust and lightweight reporting solution (often easily deployed via Docker). Through Metabase, the system will construct intelligent Dashboards, integrating visual components like trend charts, geographic distribution maps, correlation charts, along with a dynamic Slicers system. This is a powerful tool assisting leadership in easily mining detailed information about core operations: identifying top performing products, tracking cyclical revenue fluctuations, and profoundly understanding customer shopping behavior.

1.2.4. System Performance Monitoring Tools (System Monitoring)



Figure 1.8. Image of Grafana

Grafana:

- **General Description:** Grafana is a world-leading open-source platform specifically for analyzing and visualizing system monitoring metrics (observability).
- **Role in the Architecture:** If the BI layer serves business users (business analysis), then Grafana acts as a Technical Dashboard exclusively for Data Engineers. Grafana is used to monitor the "health" and performance of the entire Data Pipeline.
- **Advantages and Reasons for Selection:** Diverse connection capabilities with monitoring data sources (like Prometheus, or querying directly into ClickHouse's system log tables). Thanks to Grafana, the operations team can monitor in real-time the CPU/RAM consumption levels of Spark, execution times of DAGs in Airflow, query processing bandwidth of ClickHouse, while setting up an Alerting system immediately via email or Slack whenever any node encounters a problem.

1.3. Overview of Business Intelligence and Data Warehouse

Business Intelligence (BI) is an overarching term encompassing architectures, tools, databases, analytical applications, and methodologies aimed at supporting the corporate decision-making process. The ultimate goal of BI is to transform massive volumes of raw data into information, deduce knowledge from that information, and ultimately yield strategic actions (Actionable Insights). The heart of any BI system is the Data Warehouse, a consolidated storage platform that provides a "Single Source of Truth" for the entire organization.

1.3.1. Overall Architecture of a BI System

A complete BI system does not operate in isolation but is a complex data supply chain (Data Pipeline). This architecture is divided into 4 isolated yet closely intertwined functional layers:

- **Data Sources Layer:** This is the origin of the data ecosystem. For e-commerce systems, data is often fragmented and exists in multiple formats: structured data from sales management systems (RDBMS/OLTP), unstructured or semi-structured data from system logs (MongoDB, JSON), or independent data files (CSV, Excel). The characteristic of this layer is that data is constantly fluctuating and optimized for write speed (to avoid disrupting the customer's shopping experience), rather than for analysis.
- **Data Integration / ETL Layer:** Considered the "laboratory" of the system, where the ETL (Extract - Transform - Load) process is performed. Data from multiple sources is extracted and brought into a temporary storage area (Staging Area). Here, data engineers clean (processing garbage data, null values), standardize (unifying date and currency formats), and apply business rules (Business Rules). This is the most labor-intensive layer (often taking up 70% of a BI project's workload) aimed at ensuring data integrity and quality.
- **Data Warehouse / OLAP Layer:** A Data Warehouse differs entirely from operational databases (OLTP) in 4 core characteristics as defined by Bill Inmon:
 - **Subject-Oriented:** Data revolves around business subjects (such as Sales, Customers) instead of system applications.
 - **Integrated:** Data from heterogeneous sources is standardized into a unified format.
 - **Time-Variant:** Every record is tied to a historical time marker, allowing trend comparisons across years/months.
 - **Non-Volatile:** Data, once loaded into the Warehouse, will not be deleted or modified daily, ensuring consistency for historical reports.
- **Presentation / Analytics Layer:** The layer that communicates directly with end-users through BI tools (such as Metabase, Power BI). Complex data volumes are transformed into Dashboards, interactive reports, and visual charts, assisting leadership in easily tracking KPIs and detecting anomalies in business.

1.3.2. Importance of Dimensional Modeling

If the 4-layer architecture is the "hardware" of a BI system, then Dimensional Modeling (pioneered by Ralph Kimball) is the "soul" of the Data Warehouse. Instead of normalizing data (Normalization) to avoid duplication like traditional databases,

this methodology intentionally denormalizes (Denormalization) and divides data into two core types of tables:

- Fact Table: Stores measurable business events (like a successfully paid invoice). This table contains quantitative measures (Measures/Metrics) such as: Sales Quantity, Revenue, Discount Amount.
- Dimension Table: Contains descriptive attributes (Textual Attributes) providing the "Who, What, Where, When" context for the Fact Table. Dimension tables often contain hierarchies (Hierarchies), for example, the Time dimension has Year → Quarter → Month → Day.

Applying Dimensional Modeling (specifically the Star Schema) brings outstanding advantages:

- Optimizing OLAP Performance: By designing a central Fact Table surrounded by Dimension Tables, the system minimizes complex JOIN operations across multiple intermediate tables. This helps the system retrieve millions of records and perform aggregate calculations in just seconds.
- Business Intuitiveness: This model accurately reflects how humans think about business. When a manager wants to know "Sales of product A in region B during quarter C", the BI system simply takes "Sales" from the Fact Table and filters it through three corresponding Dimension Tables (Product, Region, Time).
- Flexible Data Exploration Capability (Slicing, Dicing & Drill-down): A multidimensional model is the foundation that allows BI tools to perform advanced analytical operations:
 - Drill-down: Going from high-level aggregated data down to detail (e.g., Drilling down from Yearly Revenue to individual Monthly Revenue).
 - Roll-up: Grouping detailed data into an aggregate whole.
 - Slice & Dice: Filtering and analyzing data by one or more specific slices (e.g., Only viewing revenue from Female customers paying with credit cards).
- Scalable Agility: When businesses have new analysis requirements, data engineers can easily add a new Dimension Table or add attribute fields to an existing table without breaking the structure of older reports.

1.4. Overall Architecture of the Data Pipeline

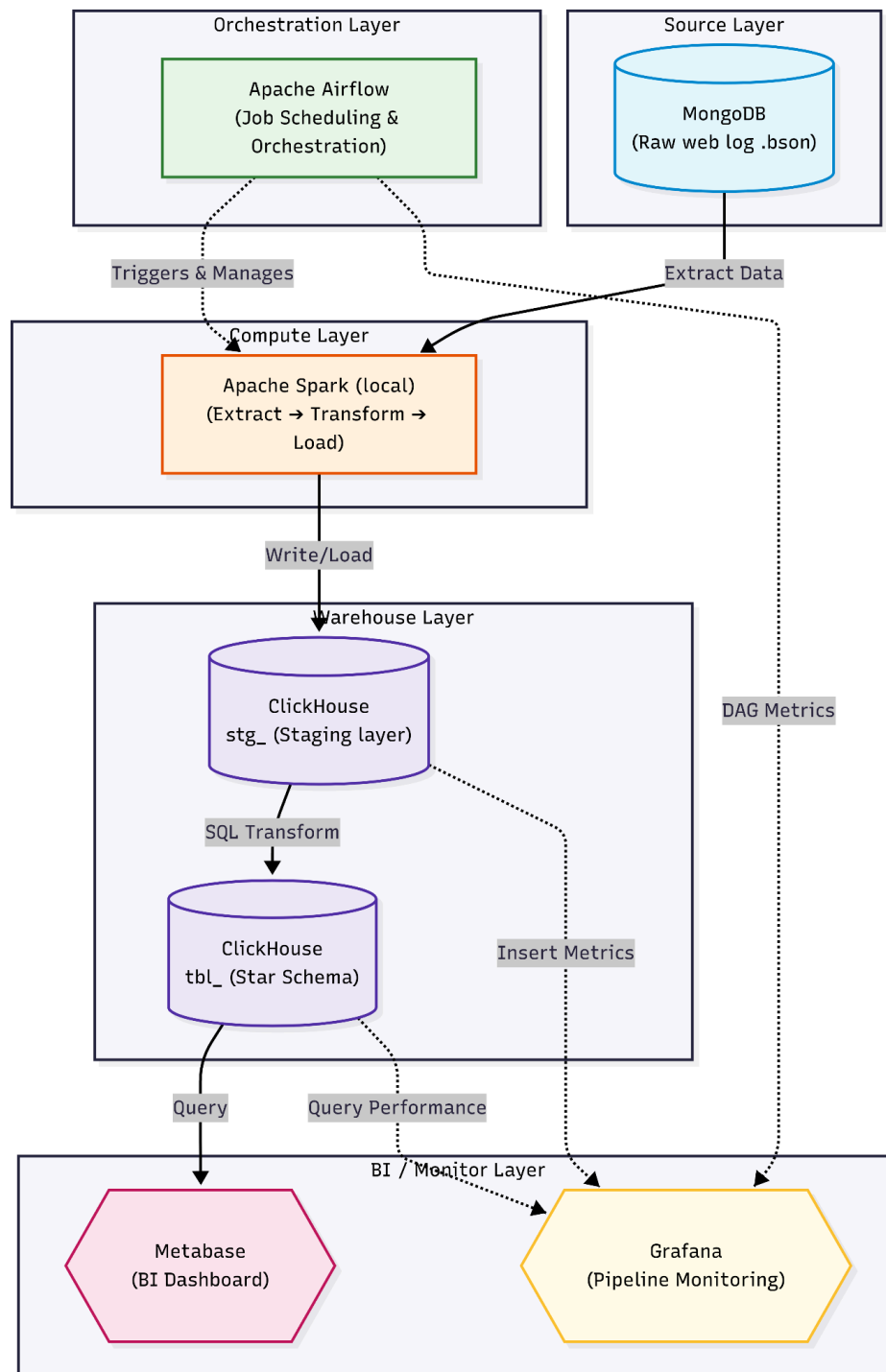


Figure 1.9. Overall Data Pipeline Architecture

To handle the immense volume of e-commerce transaction data while ensuring low latency and high reliability, the system architecture is designed based on a unidirectional data flow model. This architecture is strictly decoupled into five isolated functional layers. This separation of concerns not only ensures high

scalability but also facilitates easier maintenance, debugging, and future technology migrations. The five core layers are defined as follows:

- 1. Source Layer (Data Landing): This layer acts as the genesis of the data pipeline. The original user web log data, generated from the e-commerce platform's front-end and back-end events, is captured and stored in MongoDB. The data is retained in its native .bson format, preserving the raw, semi-structured nature of the events (e.g., nested arrays for shopping carts, varying attributes for different user actions). This ensures no data is lost at the point of ingestion and provides a flexible landing zone capable of handling high-throughput write operations.
- 2. Orchestration Layer (Workflow Management): Operating as the "conductor" of the entire system, Apache Airflow resides in this layer. It is responsible for defining, scheduling, and executing the Data Pipeline through Directed Acyclic Graphs (DAGs). Airflow triggers the downstream Spark jobs based on predefined chronological intervals (e.g., hourly or daily batches). Furthermore, it provides critical fault-tolerance mechanisms, such as automatic retries upon failure and dependency mapping, ensuring that data is processed in the correct order without manual intervention.
- 3. Compute Layer (Data Processing & ETL): This layer serves as the heavy-lifting computational engine of the pipeline, powered by Apache Spark running in local mode. Spark is tasked with the complete Extract, Transform, Load (ETL) lifecycle. It extracts the raw .bson data from MongoDB, performs complex transformations in-memory, such as data cleansing, null handling, type casting, and flattening nested JSON arrays (e.g., exploding the `cart_products` array), and finally loads the structured data into the downstream data warehouse. Leveraging Spark allows the system to efficiently process massive datasets (over 41 million records) utilizing local machine resources optimally.
- 4. Warehouse Layer (Data Storage & Modeling): This layer utilizes ClickHouse, a blazing-fast columnar database, implementing a multi-hop (Medallion-like) data architecture to guarantee data quality and analytical performance. It is subdivided into two distinct zones:
 - Staging Zone (`stg_`): Receives the newly transformed data directly from Spark. It acts as an intermediate buffer containing clean, structured data that still closely mirrors the original transactional granularity.
 - Star Schema Zone (`tbl_`): Data from the Staging zone is further transformed and distributed into a rigorous Dimensional Model, comprising a central Fact table (`tbl_fact_orders`) and surrounding Dimension tables (`tbl_dim_customer`, `tbl_dim_product`, etc.). This zone

is highly optimized for complex OLAP (Online Analytical Processing) queries.

- 5. BI & Monitor Layer (Presentation and Observability): The final layer provides interfaces for both business stakeholders and technical engineers:
 - Business Intelligence: Metabase connects directly to the Star Schema (tbl_) zone in ClickHouse. It translates the dimensional data into interactive, self-service business dashboards, allowing stakeholders to extract insights via drag-and-drop features without writing SQL.
 - System Monitoring: Grafana serves as the technical observability platform. It continuously tracks operational metrics across the pipeline, monitoring DAG execution status and durations from Airflow, the volume of rows inserted per batch, and the query performance/CPU utilization of ClickHouse. This ensures proactive detection of bottlenecks or system failures.

CHAPTER 2: DATA MODEL ANALYSIS AND DESIGN

At the core of any effective Business Intelligence system lies a well-structured and highly optimized data model. This chapter delves into the comprehensive analysis of the raw e-commerce dataset, identifying critical data quality issues and structural inconsistencies. Addressing these challenges, the chapter subsequently details the architectural design of the Data Warehouse using Dimensional Modeling principles. It outlines the creation of the Staging Layer (ODS) and the implementation of a Fact Constellation Schema (Galaxy Schema) within ClickHouse, ensuring the data is perfectly structured for high-performance analytical queries.

2.1. Input Data Profiling

2.1.1. Dataset Description

a. Sample Data

```
{
  _id: ObjectId('5ed8c9b634103036e28df298'),
  time_stamp: 1591265719,
  ip: '109.208.234.125',
  user_agent: 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14) AppleWebKit/605.1.15',
  resolution: '1680x1050',
  user_id_db: '',
  device_id: 'f059f067-ff73-44cd-9a7d-a20173f817b4',
  api_version: '1.0',
  store_id: '12',
  local_time: '2020-06-04 12:15:15',
  show_recommendation: null,
  current_url: 'https://www.glamira.fr/glamira-bagues-gratia.html?alloy=red_white',
  referrer_url: '',
  email_address: '',
  recommendation: false,
  utm_source: false,
  utm_medium: false,
  collection: 'view_product_detail',
  product_id: '90736',
  option: [
    {
      option_label: 'alloy',
      option_id: '326941',
      value_label: '',
      value_id: '3176136'
    },
    {
      option_label: 'diamond',
      option_id: '326940',
      value_label: '',
      value_id: '3176099'
    }
  ]
},
```

Figure 2.1. Sample data

b. All Data Fields

```
{ field: '_id', types: ['objectId'] }
{ field: 'ip', types: ['string'] },
{ field: 'email_address', types: ['null', 'string'] },
{ field: 'currency', types: ['string', 'null'] },
{ field: 'order_id', types: ['int', 'double', 'string'] },
{ field: 'utm_source', types: ['string', 'null', 'bool'] },
{ field: 'collection', types: ['string'] },
{ field: 'collect_id', types: ['string'] },
{ field: 'cat_id', types: ['null', 'string'] },
{ field: 'recommendation_clicked_position', types: ['int', 'null'] },
{ field: 'viewing_product_id', types: ['string'] },
{ field: 'referrer_url', types: ['string'] },
{ field: 'utm_medium', types: ['bool', 'null', 'string'] },
{ field: 'key_search', types: ['null', 'string'] },
{ field: 'recommendation_product_position', types: ['int', 'string', 'null'] },
{ field: 'local_time', types: ['string', 'null'] },
{ field: 'recommendation', types: ['bool', 'null'] },
{ field: 'resolution', types: ['string', 'null'] },
{ field: 'show_recommendation', types: ['string', 'null'] },
{ field: 'user_agent', types: ['string'] },
{ field: 'recommendation_product_id', types: ['string', 'null'] },
{ field: 'device_id', types: ['string'] },
{ field: 'product_id', types: ['string'] },
{ field: 'store_id', types: ['string'] },
{ field: 'time_stamp', types: ['int'] },
{ field: 'user_id_db', types: ['string'] },
{ field: 'price', types: ['string', 'null'] },
{ field: 'is_paypal', types: ['null', 'bool'] },
{ field: 'api_version', types: ['string'] },
{ field: 'current_url', types: ['string'] },
{ field: 'option', types: ['array', 'object'] },
{ field: 'cart_products', types: ['array'] },
```

c. Detailed Data Description

- `_id` (objectId): Primary key, unique identifier for each record.
- `ip` (string): User's IP address, used to track the source of access.

- email_address (null, string): User's email, used for identification or marketing.
- currency (string, null): Currency type (USD, EUR), used for pricing and payments.
- order_id (int, double, string): Order ID, used to track transactions.
- utm_source (string, null, bool): Marketing source (Google, Facebook), used to track traffic.
- collection (string): Event type (view_product_detail, checkout_success), categorizes user behavior.
- collect_id (string): Category ID or event group, used for classification.
- cat_id (null, string): Product category ID (ring, necklace).
- recommendation_clicked_position (int, null): Position of the clicked recommended product.
- viewing_product_id (string): ID of the currently viewed product.
- referrer_url (string): The referrer URL directing the user to the website.
- utm_medium (bool, null, string): Marketing medium (CPC, email).
- key_search (null, string): Search keyword entered by the user.
- recommendation_product_position (int, string, null): Product position in the recommendation list.
- local_time (string, null): User's local time.
- recommendation (bool, null): Flag indicating an action related to recommendations.
- resolution (string, null): Device screen resolution.
- show_recommendation (string, null): Display status of the recommendation system.
- user_agent (string): User's browser/device information.
- recommendation_product_id (string, null): Recommended product ID.
- device_id (string): User device identifier.
- product_id (string): Product ID related to the action (cart, checkout).
- store_id (string): Store or branch code.
- time_stamp (int): Event time (Unix timestamp).
- user_id_db (string): User ID in the database.
- price (string, null): Product or order price.
- is_paypal (null, bool): Flag indicating a PayPal transaction.
- api_version (string): API version used to log the data.
- current_url (string): Current URL the user is visiting.
- option (array, object): Product options (size, color).
- cart_products (array): List of products in the shopping cart.

2.1.2. Identifying Raw Data Issues

The characteristic of log data collected from e-commerce websites and stored in a NoSQL format (MongoDB) is high flexibility but a lack of integrity constraints. Through preliminary Data Profiling, several severe data quality issues can be identified. If not resolved during the ETL phase, these issues will cause serious deviations in the BI reports. The core issues include:

- Missing Values / Nulls
 - Current Status: Many important data fields contain null values, typically email_address, currency, cat_id, utm_source, price, and key_search.
 - Impact: The absence of price or currency will directly skew revenue analysis. A missing utm_source reduces accuracy when analyzing user behavior across marketing channels.
 - Solution: Establish a mechanism to filter out records missing critical data (e.g., orders without prices), or apply default value imputation (e.g., "Unknown") for categorical fields to maintain referential integrity when loading data into the Data Warehouse.
- Inconsistent Data Types
 - Current Status: According to the schema description, the order_id field exists in three different formats: ['int', 'double', 'string']. The price field is stored as a string (string) or null instead of a float/decimal type.
 - Impact: BI tools and relational databases require consistent data types for each column. This discrepancy will cause system errors (Type Mismatch) when performing aggregate functions (SUM, AVG) or JOIN operations.
 - Solution: Perform Type Casting to convert all order_id values to a string format and price values to a decimal/float format during the Transform process.
- Complex Nested Data Structures
 - Current Status: The option fields (containing array/object) and especially cart_products (containing an array of products) store multiple pieces of information within a single record.
 - Impact: The Star Schema of the Data Warehouse only accepts flat data (single-value row-column format).
 - Solution: Use array processing functions (e.g., the explode function in Apache Spark) to flatten the cart_products array. Each product in the shopping cart will be separated into a distinct row, tied to the same transaction session ID.
- Time Format Issues

- Current Status: The time_stamp field is stored as an integer (int - Unix timestamp), while local_time is a string.
- Impact: To resolve the "Monthly Revenue" analysis on the Dashboard, the system needs a standard Time Dimension Table (Dim_Time). The Unix timestamp format cannot be directly analyzed by day, month, or quarter.
- Solution: Convert the time_stamp from an integer format to a standard DateTime format (e.g., YYYY-MM-DD HH:MM:SS), and simultaneously extract it into smaller sub-fields (Year, Month, Day) to facilitate linking with the Dim_Time table.
- Data Duplicates
 - Current Status: During event logging, network errors or continuous page reloads by users can create completely duplicated records regarding time_stamp, ip, user_id_db, and collection.
 - Impact: Artificially inflates the number of product views, order quantities, or total revenue.
 - Solution: Apply Deduplication mechanisms based on unique keys or a combination of fields (such as _id, time_stamp, user_id_db) to retain only the valid, unique record.
- Noise and Outliers
 - Current Status: Web log systems often record non-human traffic (like Bots, Crawlers) or test orders from the development team. This is reflected in the user_agent field or abnormal price levels.
 - Impact: Distorts metrics regarding real user behavior (User Flow) and Conversion Rates.
 - Solution: Establish rule sets to filter out records exhibiting anomalous patterns before loading them into the Fact table.

2.2. Data Warehouse Design

The Data Warehouse (DWH) for the Glamira project is designed using Dimensional Modeling principles, optimized specifically for the ClickHouse columnar database engine. The core architecture, named glamira_dw, establishes a rigorous separation between quantitative performance metrics (Fact tables) and descriptive context (Dimension tables). Foreign Keys (FK) with the _id suffix in the Fact tables directly map to Primary Keys (PK) of the same name in the corresponding Dimension tables.

2.2.1. Business Process Identification

To comprehensively capture the e-commerce dynamics of Glamira, the data warehouse design focuses on two core, distinct business processes:

1. Sales and Financial Performance: Tracking successful checkout transactions to analyze revenue, product performance, and pricing across different markets.
2. User Behavior and Clickstream: Monitoring customer interactions and navigation paths (e.g., viewing products, adding to cart) to analyze marketing traffic, conversion funnels, and store engagement.

2.2.2. Staging Layer Design (ODS - Operational Data Store)

Before data is modeled into the complex Star Schema, it first lands in the Staging Layer (prefixed with `stg_`). This layer acts as a buffer zone, storing data that closely mirrors the raw source but is structurally validated and cast into relational tables. Utilizing ClickHouse's `ReplacingMergeTree` engine, this layer effectively handles data deduplication and updates.

1. Table `stg_events` (User Clickstream Log Data)

- Description: Stores all user behaviors and interactions on the website system (e.g., viewing products, adding to cart, proceeding to checkout, successful checkout).
- Engine: `ReplacingMergeTree(event_time)` - Ensures that duplicated log events are deduplicated based on the event timestamp.
- Partition Strategy: Partitioned by day to `YYYYMMDD(event_time)` to optimize the purging of old logs and facilitate daily ETL batch loads.

Column Name	ClickHouse Data Type	Role	Description
<code>event_id</code>	String	Key	Unique identifier for each system log event.
<code>event_time</code>	<code>DateTime64(3)</code>	Key / Version	Server-side event timestamp (millisecond precision).
<code>local_time</code>	<code>Nullable(DateTime)</code>	Attribute	User's local device time when the behavior occurred.
<code>collection</code>	String	Key	Behavior type (view_product_detail, checkout_success, etc.).

ip	Nullable(String)	Attribute	User's device IP address.
user_agent	Nullable(String)	Attribute	Browser and operating system info string.
resolution	Nullable(String)	Attribute	Device screen resolution (e.g., 375x667).
user_id_db	Nullable(String)	Attribute	Logged-in user ID (stored as raw string).
device_id	Nullable(String)	Attribute	Anonymous device identifier (Cookie/Session Token).
store_id	Nullable(String)	Attribute	Web store version/country code (e.g., Store Germany).
email_address	Nullable(String)	Attribute	Customer's email address (if provided during checkout).
current_url	Nullable(String)	Attribute	The current URL the user is interacting with.
referrer_url	Nullable(String)	Attribute	The previous URL that directed the user to the current page.
utm_source	Nullable(Bool)	Attribute	Flag indicating paid ad source (True: Paid / False or Null).
utm_medium	Nullable(Bool)	Attribute	Flag indicating the paid marketing medium.

key_search	Nullable(String)	Attribute	Keyword searched by the user on the website.
api_version	Nullable(String)	Attribute	API version recording the backend log.
product_id	Nullable(Int32)	Attribute	Interacted product ID.
cat_id	Nullable(String)	Attribute	Product Category ID.
collect_id	Nullable(String)	Attribute	Product Collection ID.
order_id	Nullable(String)	Attribute	Generated order ID (only present when collection = 'checkout_success').
price	Nullable(Decimal(18, 2))	Attribute	Single product price stored in the system.
currency	Nullable(String)	Attribute	Raw display currency (e.g., EUR, USD, VND).
is_paypal	Nullable(Bool)	Attribute	Flag verifying if payment was made via PayPal.
recommendation	Nullable(Bool)	Attribute	Marks if the product view was driven by AI recommendation.
show_recommendation	Nullable(String)	Attribute	Display status of the recommendation block ("true"/"false").
recommendation_product_id	Nullable(String)	Attribute	ID of the product suggested by the AI system.
recommendation_product_position	Nullable(Int32)	Attribute	Display position of the suggested product in the list.

recommendation_clicked_position	Nullable(Int32)	Attribute	Actual position of the suggested product clicked by the user.
option_json	Nullable(String)	Attribute	JSON string storing jewelry attributes (Alloy, Diamond, Size).
cart_products_json	Nullable(String)	Attribute	JSON string containing the array of products in the cart upon successful checkout.

2. Table stg_products (Raw Product Catalog Data)

- Description: Contains basic catalog information regarding the product SKUs currently sold on the Glamira system.
- Engine: ReplacingMergeTree(updated_at) - Automatically updates to the latest product name based on the product ID.

Column Name	ClickHouse Data Type	Role	Description
product_id	Int32	Primary Key	Unique product identifier.
product_name	String	Attribute	Detailed commercial name of the product.
updated_at	DateTime	Version	Timestamp when the product data was updated in staging.

3. Table stg_country (International Standard Country Mapping)

- Description: A geographical reference table based on the ISO 3166 standard, used to standardize the Top-Level Domain (TLD) data extracted from URLs.
- Engine: ReplacingMergeTree(updated_at)

Column Name	ClickHouse Data Type	Role	Description
-------------	----------------------	------	-------------

name	String	Attribute	Full country name in English.
alpha_2	String	Attribute	2-letter ISO standard abbreviation (e.g., DE, FR).
alpha_3	String	Attribute	3-letter ISO standard abbreviation (e.g., DEU, FRA).
country_code	String	Primary Key	National phone code or region identifier.
iso_3166_2	String	Attribute	International standard sub-region code.
region	String	Attribute	Representative continent name (e.g., Europe, Asia).
sub_region	String	Attribute	Smaller continental sub-region.
intermediate_region	Nullable(String)	Attribute	Detailed intermediate geographical zone.
region_code	String	Attribute	Continent identifier code.
sub_region_code	String	Attribute	Continental sub-region identifier code.
intermediate_region_code	Nullable(String)	Attribute	Intermediate zone identifier code.

updated_at	DateTime	Version	Timestamp of system information logging.
------------	----------	---------	--

2.2.3. Fact Table Design

To accurately reflect the two identified business processes without causing data overlap or grain mismatch, the system implements two separate Fact tables.

1. Bảng fact_order (Sales & Detailed Order Fact)

- Description: Stores highly detailed data of successful order transactions (checkout_success). The data granularity is at the deepest level: one row represents a single, independent product within an order's shopping cart.
- Database Engine: MergeTree()
- Physical Sorting Strategy (ORDER BY): Clustered by (date_id, territory_id, product_id). This strictly optimizes computation speeds for revenue reports segmented by time, national markets, and product performance.

Column Name	ClickHouse Data Type	Role	Description
event_id	String	DD-Key	Original log event ID generating this successful order.
order_id	String	DD-Key	Commercial invoice ID of the order.
date_id	Int64	Foreign Key	Links to dim_date (Time of purchase).
territory_id	Int64	Foreign Key	Links to dim_territory (Purchasing market).
device_id	String	Foreign Key	Links to dim_device (Device used).

product_id	Int64	Foreign Key	Links to dim_product (Purchased product).
currency_id	Int64	Foreign Key	Links to dim_currency (Payment currency).
alloy_id	Int64	Foreign Key	Links to dim_material (Base metal of product).
diamond_id	Int64	Foreign Key	Links to dim_material (Attached gemstone).
user_id_db	Nullable(Int64)	Attribute	Member account ID (Null for guest checkouts).
show_recommendation	UInt8	Attribute	Binary flag (1: AI-suggested purchase / 0: Organic purchase).
amount	Int32	Metric	Quantity of the specific product ordered.
price_local	Float64	Metric	Unit price in local currency at the respective store.
price_usd	Nullable(Float64)	Metric	Standardized unit price converted to US Dollars (USD).

2. Bảng fact_events (Aggregated Clickstream Behavior Fact)

- Description: Stores all customer action sequences, interactions, and navigation paths to facilitate Conversion Funnel and Marketing Traffic analysis. This table

explicitly excludes checkout_success events to prevent data duplication with fact_order.

- Database Engine: MergeTree()
- Physical Sorting Strategy (ORDER BY): Clustered by (date_id, collection, store_id) to optimize queries counting hourly events, conversion behavior funnels, and store-specific traffic.

Column Name	ClickHouse Data Type	Role	Description
event_id	String	DD-Key	Unique ID of the behavioral event.
collection	String	Attribute	Interaction type (e.g., view_product_detail, add_to_cart).
date_id	Int64	Foreign Key	Links to dim_date (Time of event).
territory_id	Int64	Foreign Key	Links to dim_territory (Market origin).
device_id	String	Foreign Key	Links to dim_device (Interacting device).
store_id	String	Attribute	Web store branch code recording the event.
user_id_db	Nullable(String)	Attribute	User ID string if the action occurs post-login.
product_id	Int64	Foreign Key	Links to dim_product (Viewed/interacted product).
current_url	String	Attribute	Raw URL where the event occurred (for deep funnel analysis).
utm_source	String	Attribute	Marketing source driving the traffic (e.g., Organic, Paid).
utm_medium	String	Attribute	Marketing distribution method.

event_count	Int32	Metric	Standardized count (Default = 1) for rapid SUM() aggregations.
-------------	-------	--------	--

2.2.4. Dimension Table Design

Dimension tables provide the descriptive context for the business events. These tables are engineered to handle dynamic JSON attributes and global scaling.

1. Bảng dim_date (Time Analysis Dimension)

- Description: Auto-generated time dimension with granularity down to the unique hour. Covers an extensive range for chronological trend analysis.
- Engine: MergeTree()
- PK Generation Rule: Integer format HHddMMyyyy (e.g., 14:00 on 02/06/2026 becomes 1402062026).

Column Name	Data Type	Role	Description
date_id	Int64	Primary Key	Unique time identifier per hour (HHddMMyyyy).
full_date	DateTime	Attribute	Full standard DateTime value.
day_of_week	String	Attribute	Full weekday name (e.g., Monday).
day_of_week_short	String	Attribute	3-letter weekday abbreviation (e.g., Mon).
day_of_month	Int32	Attribute	Day of the month (1 to 31).
month	Int32	Attribute	Month of the year (1 to 12).
year	Int32	Attribute	4-digit analytical year.
hour	Int32	Attribute	Hour of the day (0 to 23).

2. Bảng dim_territory (Geospatial & Market Dimension)

- Description: Manages Glamira's national market data, parsed and hashed from the Top-Level Domain (TLD) of the current_url.
- Engine: MergeTree()

- PK Generation Rule: Absolute hash of the country domain string `abs(hash(country_code))`. Record -1 represents the global market (glamira.com) or unhashable URLs.

Column Name	Data Type	Role	Description
territory_id	Int64	Primary Key	Hashed identifier of the national market.
country_code	String	Attribute	Website TLD abbreviation (de, fr, au, com).
country_name	Nullable(String)	Attribute	Standardized country name from ISO table (e.g., Germany).
alpha_2	Nullable(String)	Attribute	2-letter uppercase ISO country code (DE, FR).
alpha_3	Nullable(String)	Attribute	3-letter uppercase ISO country code (DEU, FRA).
region	Nullable(String)	Attribute	Major continent name (e.g., Europe, Asia).
sub_region	Nullable(String)	Attribute	Continental sub-region.
intermediate_region	Nullable(String)	Attribute	Detailed intermediate regional zone.

3. Bảng dim_product (Product Information Dimension)

- Description: Contains the catalog of products for report rendering via JOIN operations.
- Engine: MergeTree()

Column Name	Data Type	Role	Description
-------------	-----------	------	-------------

product_id	Int64	Primary Key	Unique product identifier.
product_name	Nullable(String)	Attribute	Commercial display name of the jewelry product.

4. Bảng dim_device (Device & Browser Dimension)

- Description: Manages customer access environment details. OS, Browser, and Device Type are scientifically parsed from the raw user_agent string.
- Engine: ReplacingMergeTree() - Automatically overwrites and updates if the configuration of the same device_id changes.

Column Name	Data Type	Role	Description
device_id	String	Primary Key	Unique device identification token.
user_agent	Nullable(String)	Attribute	Full raw User-Agent string from system logs.
os	Nullable(String)	Attribute	Parsed Operating System (e.g., Windows, iOS).
browser	Nullable(String)	Attribute	Executing web browser (e.g., Chrome, Safari).
device_type	Nullable(String)	Attribute	Device category (Mobile / Tablet / Desktop).
resolution	Nullable(String)	Attribute	Raw screen display resolution (e.g., 1920x1080).

5. Bảng dim_material (High-End Jewelry Material Dimension)

- Description: A smart consolidated table storing product composition data, encompassing both Base Metals (Alloy) and Attached Gemstones (Diamond), as they share identical attribute structures.
- Engine: MergeTree()
- PK Generation Rule: Calculated via hash of the material display label to synchronize dynamic string analysis from JSON: abs(hash(value_label)).

Column Name	Data Type	Role	Description
material_id	Int64	Primary Key	Hashed identifier of the attribute material.
material_type	String	Attribute	Independent material category (alloy or diamond).
value_label	String	Attribute	Detailed material display label (e.g., Gelbgold 585).

6. Bảng dim_currency (International Exchange Rate Dimension)

- Description: Manages the catalog of transactional currencies and their financial conversion rates to USD for standardized corporate revenue reporting.
- Engine: MergeTree()
- PK Generation Rule: abs(hash(currency_code))

Column Name	Data Type	Role	Description
currency_id	Int64	Primary Key	Unique hashed currency identifier.
currency_code	String	Attribute	Standard financial currency abbreviation (EUR, USD).
usd_conversion_rate	Nullable(Float64)	Attribute	Direct multiplier to convert local currency to USD.

2.2.5. Schema Selection

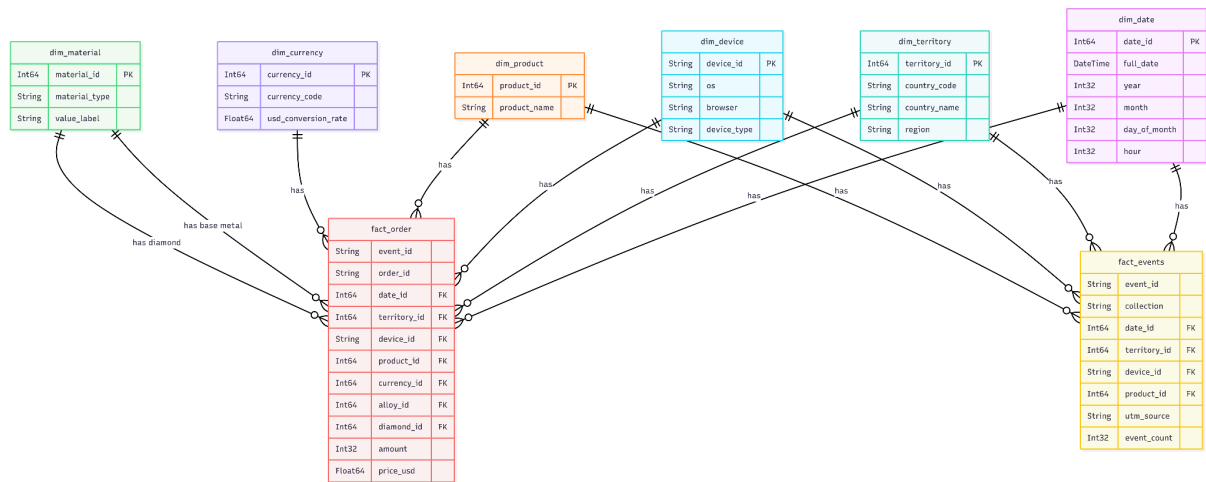


Figure 2.2. Fact Constellation Schema (Galaxy Schema).

The system implements a Fact Constellation Schema (often referred to as a Galaxy Schema), which is an advanced evolution of the traditional Star Schema.

- **Architectural Fit:** This schema was chosen because the business requirements dictate tracking multiple distinct processes (Sales via `fact_order` and Traffic/Clickstream via `fact_events`).
- **Conformed Dimensions:** Both fact tables share common dimensions, such as `dim_date`, `dim_territory`, `dim_device`, and `dim_product`. This sharing of "Conformed Dimensions" ensures that business users can cross-analyze data (e.g., comparing the number of product views from `fact_events` against the actual product revenue from `fact_order` within the same geographical territory and time frame).
- **ClickHouse Optimization:** The denormalized nature of the dimension tables connected directly to the fact tables avoids multi-level cascading JOINS (which are required in Snowflake schemas). This perfectly aligns with ClickHouse's columnar architecture, ensuring blazing-fast read and aggregation speeds across massive datasets when querying from the BI tool (Metabase).

CHAPTER 3: ETL PROCESS IMPLEMENTATION AND DATA CLEANSING

In any Business Intelligence system, the quality of the output reports is strictly dependent on the integrity of the input data, adhering to the "Garbage In, Garbage Out" (GIGO) principle. Given the nature of website log data stored in MongoDB, which boasts high structural flexibility but often contains dirty data, noise, and formatting inconsistencies, the project utilized Apache Spark (specifically PySpark) to architect a robust ETL (Extract, Transform, Load) pipeline. This pipeline facilitates the seamless processing of a massive data volume (over 41 million records) entirely on local machine resources without compromising performance.

3.1. Data Extraction (Extract)

The process of extracting data from the MongoDB source database (the summary collection) required specialized configurations to prevent Out-Of-Memory (OOM) errors and pipeline crashes caused by heterogeneous data types.

- **Defensive Schema-on-Read Definition:** Relying on Spark's default Schema Inference is highly risky for NoSQL data, as a single field might contain a mix of strings, floats, and nulls across millions of documents, leading to fatal Type Mismatch errors. To mitigate this, the project predefined a strict RAW_SCHEMA. All 32 input fields were initially ingested as strings (StringType()). The actual Type Casting was intentionally deferred and tightly controlled in the subsequent Transform phase.
- **Hardware Resource Optimization:** The Spark Session was configured to run in local[*] mode, compelling the engine to fully utilize all available CPU cores (multi-threading). The system was allocated 22GB of RAM. To prevent driver memory bottlenecks, the extraction was optimized using the MongoDB Connector's SamplePartitioner. By capping the partition size at 64MB, Spark could read the 41 million records in small, highly parallelized batches, multiplying the extraction throughput significantly.

3.2. Data Transformation and Cleansing (Transform)

This is the most complex phase of the ETL pipeline, comprising 12 consecutive transformation steps executed via PySpark's DataFrame API.

3.2.1. Handling Errors and Missing Values

- **Whitespace Trimming and Empty String Standardization:** An iterative loop was applied across all 22 string-based columns (e.g., `ip`, `user_agent`, `email_address`) to trim trailing and leading whitespaces. Any resulting empty strings ("") were immediately converted to standard SQL NULL values. This not only optimizes storage space but also ensures mathematical accuracy for aggregate functions (like COUNT) in the data warehouse.
- **Boolean Logic Standardization:** Several fields, such as `utm_source`, `utm_medium`, and `is_paypal`, contained chaotic, mixed-case logical values (e.g., "True", "true", "1", "0", "false"). A custom `cast_to_boolean` function was applied to map these disparate values uniformly into standard BooleanType or ByteType (0 and 1) formats.
- **Data Quality Validation (Circuit Breaker):** Before the data is allowed to be loaded, a Single-pass Aggregation is triggered to detect anomalies without forcing expensive data shuffles. The system counts and automatically issues warnings (or halts the job) if it detects fatal flaws: null event IDs (`event_id`), missing timestamps (`event_time`), or mathematically impossible negative prices (`price < 0`).

3.2.2. Data Standardization

- **Identifier (ID) Formatting:** The `_id` field originating from MongoDB was nested within a JSON object (e.g., `{"_id": "..."}`). The `get_json_object` function was employed to extract the actual ID string. Similarly, the `order_id` field, which suffered from floating-point string artifacts (e.g., "12345.0"), was cleansed using Regular Expressions (Regex) to return a clean, standard integer string.
- **Time Formatting:** The raw `time_stamp` (a 10-digit Unix epoch integer) was reverse-engineered into a standard TimestampType (`event_time`). For the `local_time` field, which exhibited severe format fragmentation (containing 'T' separators, timezones, etc.), the system utilized the `coalesce` function to scan through a hierarchy of DateTime patterns. Specifically, the pattern utilized a single 'H' (instead of 'HH') to accurately capture single-digit hours.
- **Currency and Price Precision:** The price field was strictly cast from a string to a DecimalType(18, 2). Utilizing the Decimal type, rather than Float or Double, is a mandatory practice in financial data engineering to prevent floating-point arithmetic rounding errors when calculating total revenue on the BI Dashboard.

3.2.3. Restructuring and Derived Metrics

- **Field Coalescing:** The system identified that user product-viewing events stored the product identifier in either the `product_id` or `viewing_product_id` field, depending on the user's behavior (organic click vs. AI recommendation click). A new column was derived to merge (coalesce) these two fields, ensuring the behavioral tracking flow remained unbroken.
- **Handling Nested JSON Arrays:** The `option` and `cart_products` fields contained highly complex arrays. *Crucially*, instead of attempting to immediately flatten (Explode) these arrays within Spark, which would cause massive data multiplication and inevitably crash the local machine's RAM, an optimized architectural decision was made. The JSON strings were merely cleansed at this stage (e.g., renaming "category id" to "category_id", wrapping loose {} in array brackets [{}]), and cast as valid JSON strings (e.g., `cart_products_json`). The computationally heavy task of parsing and exploding this JSON array was intentionally deferred to ClickHouse's powerful analytical engine in the final loading phase.

3.3. Loading Data into the Data Warehouse (Load)

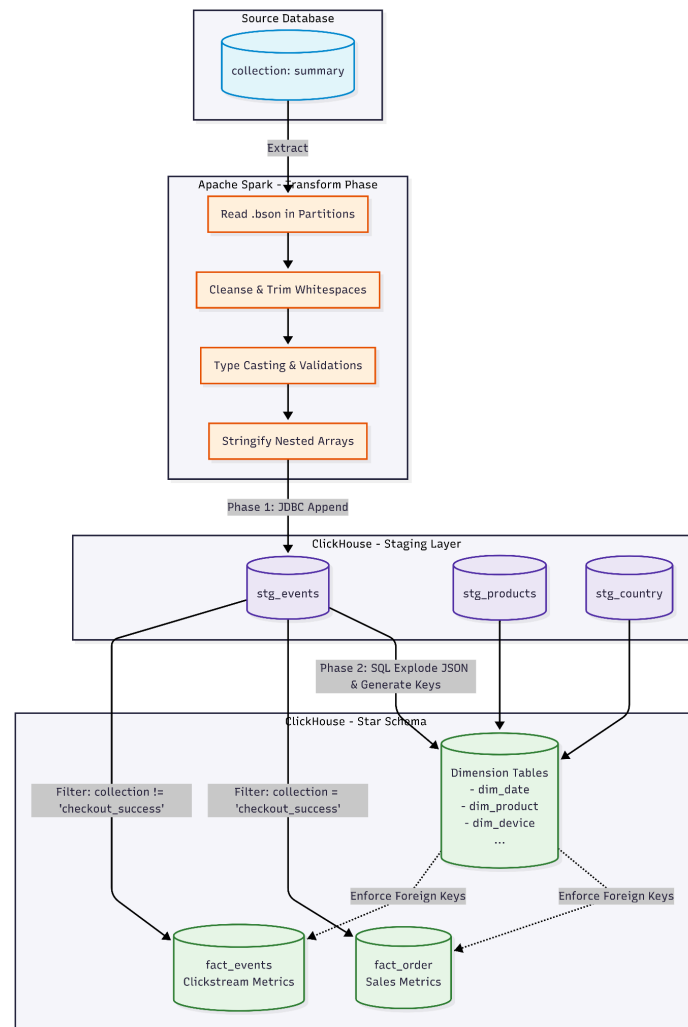


Figure 3.1. Detailed ETL Data Flow from Staging to Star Schema.

Adhering to the designed multi-hop system architecture, the Data Loading procedure was divided into two independent, sequential stages to guarantee the safety and integrity of the core analytical data warehouse:

- **Phase 1: Ingestion into the Staging Layer (Executed by Spark)** After passing through Spark's transformation logic, the semi-processed dataframe is pushed into the `stg_events` table in ClickHouse via the JDBC protocol. To prevent network bottlenecks when writing 41 million records, the DataFrame is dynamically repartitioned (`repartition(32)`) into 32 smaller data chunks. Spark executes the write operation in batches (`batchsize="20000"`) combined with the `rewriteBatchedStatements="true"` configuration to maximize ClickHouse's INSERT performance. The write mode is set to Append. Once ingested,

ClickHouse's native ReplacingMergeTree table engine autonomously handles data deduplication based on the event_time versioning key.

- **Phase 2: Transformation into the Star Schema (Orchestrated by Airflow / Executed by ClickHouse SQL)** From the stg_events buffer zone, an internal ClickHouse SQL process (scheduled and orchestrated by Apache Airflow) is triggered. This downstream process reads the cart_products_json strings and leverages ClickHouse's native, highly-optimized JSON processing functions to Explode the cart items into individual rows. The data is then mapped to generate Surrogate Keys. To strictly enforce Referential Integrity, the data is INSERTed into the Dimension tables (dim_customer, dim_product, dim_date, etc.) first. Finally, the standardized quantitative metrics are loaded into the Fact tables (fact_orders and fact_events), officially completing the Dimensional Model and rendering it ready for high-speed querying via Metabase.

CHAPTER 4: DASHBOARD CONSTRUCTION AND INSIGHTS ANALYSIS

The ultimate objective of a Business Intelligence pipeline is to transform structured data into actionable visual insights. This chapter focuses on the Presentation Layer of the project, showcasing the interactive dashboards constructed using Metabase. It provides an in-depth analysis of three critical business dimensions: top-selling product performance, historical revenue trends with seasonal patterns, and customer purchasing behavior across diverse geographical regions. Finally, the chapter concludes with an evaluation of the BI system's overall effectiveness, highlighting both its strengths and technical limitations.

4.1. Dashboard Design Architecture

4.1.1. Structure and User Interface (UI)

To optimize the User Experience (UX) and minimize the cognitive load for business managers and executives, the Metabase dashboard system is strictly organized into a hierarchical structure consisting of four specialized functional tabs. This modular approach ensures a clear separation between Financial data streams (Sales/Revenue) and Digital Behavioral data streams (Clickstream/Traffic):

- **Tab 1: Revenue Overview & Business Performance:** Serves as the executive summary, providing core financial health indicators (KPIs) and macro-level operational trends over time.
- **Tab 2: Geographical Business Analysis:** Visualizes the global flow of revenue through spatial mapping, directly supporting regional logistics and localized marketing campaigns.
- **Tab 3: Product & Material Performance:** Drills down into the granular structure of the jewelry catalog, analyzing the profitability of specific alloys, diamonds, and their combinations.
- **Tab 4: Traffic Landscape & User Behavior Funnel:** Shifts the focus to product analytics, evaluating the website's technical performance and tracking user drop-off rates through the conversion funnel.

The system integrates intelligent Global Filters at the top of each page, allowing users to dynamically slice the data across temporal dimensions (Year/Month/Day) and spatial dimensions (Country/Continent). Upon applying a filter, all visualizations within the tab synchronize in real-time, leveraging the highly optimized distributed querying capabilities of the underlying ClickHouse database.

4.1.2. Metric Calculations and Measure Generation

Because Metabase does not utilize DAX (Data Analysis Expressions) like Power BI, advanced metrics and calculated fields are realized through a hybrid approach utilizing two core techniques:

- **Data-Tier Optimization (SQL Data Mart Views):** Computationally heavy operations, such as timezone normalization, timestamp extraction (hour/day parsing), and spatial string concatenations, are pushed down to the database level. These are permanently materialized or compressed into views like `glamira_dm.view_sales_performance` and `glamira_dw.v_behavior_detail` to guarantee lightning-fast retrieval.
- **Metabase Custom Columns & Aggregations:** For dynamic indicators that must recalculate based on user-applied filters, the system utilizes Custom Expressions directly within the Metabase UI. For example, Average Order Value (AOV) is calculated as `Sum(price_usd) / Distinct(order_id)`, and Feature Conversion Rate is calculated as `CountIf(event_action == 'select_product_option') / CountIf(event_action == 'view_product_detail')`.

4.2. Revenue and Business Performance Analysis (Tab 1)

The first tab acts as the "Command Center," providing a holistic view of cash flow magnitude and the commercial heartbeat of the enterprise over time.

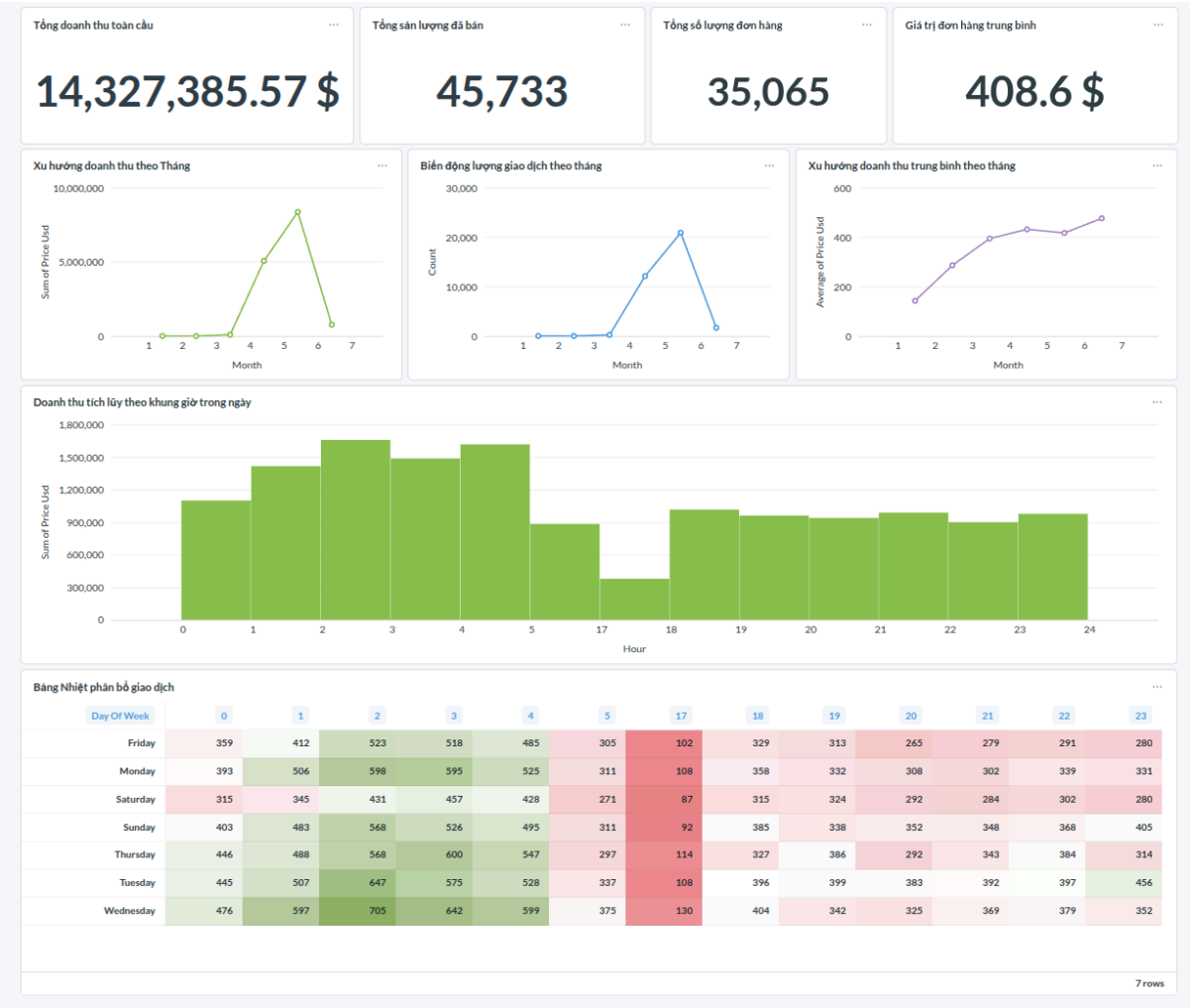


Figure 4.1. Revenue and Business Performance Dashboard (Tab 1)

4.2.1. Strategic Overview Indicators (KPI Cards)

Large KPI Cards are positioned at the topmost section of the dashboard to deliver an immediate financial snapshot:

Report Name	Chart Type	Metric Formula	Business Significance
Total Revenue (USD)	Number / KPI	Sum(price_usd)	Measures the absolute scale of incoming cash flow, standardized in USD for global consistency.

Total Orders	Number / KPI	Distinct count of order_id	Represents total successful transactions, reflecting market reach and customer acquisition volume.
Items Sold	Number / KPI	Sum(amount)	Indicates the actual physical volume of goods leaving the warehouse (inventory turnover).
Average Order Value (AOV)	Number / KPI	Sum(price_usd) / Distinct(order_id)	Reflects consumer purchasing power and measures the effectiveness of upselling and cross-selling strategies.

4.2.2. Time-Series Insights and Revenue Trends

The system employs continuous time-series visualizations to isolate seasonality and temporal shopping behaviors:

- **Revenue & Order Trends by Month (Dual-axis Line Chart):** Illustrates the linear correlation between order volume and total revenue. This clearly exposes seasonality, identifying sales spikes during Q4 holiday events (Black Friday, Christmas) or slumps during off-peak summer months.
- **AOV Trend Analysis (Line Chart):** Tracks whether the value of the average shopping cart is expanding or contracting over time, allowing managers to assess if promotional bundling campaigns are working.
- **Cumulative Revenue by Hour (Bar Chart):** Aggregates incoming revenue across a 24-hour axis, pinpointing the "golden hours" when transactions peak daily.
- **Order Distribution Heatmap (Matrix Table):** A two-dimensional matrix combining Day of the Week and Hour of the Day. Cells with darker conditional formatting indicate precise moments of high buyer intent (e.g., Sunday evenings or Friday noons).
- **Actionable Insight:** Revenue generation is highly concentrated during specific timeframes and exhibits sharp growth during event cycles. By leveraging the "Golden Hour" Heatmap, the performance marketing team can automate

bidding strategies to aggressively scale ad spend during high-conversion windows, thereby maximizing Return on Ad Spend (ROAS).

4.3. Geographical Business Analysis (Tab 2)

This interface is entirely dedicated to spatial analytics and international market profiling, helping to identify core demographic targets.

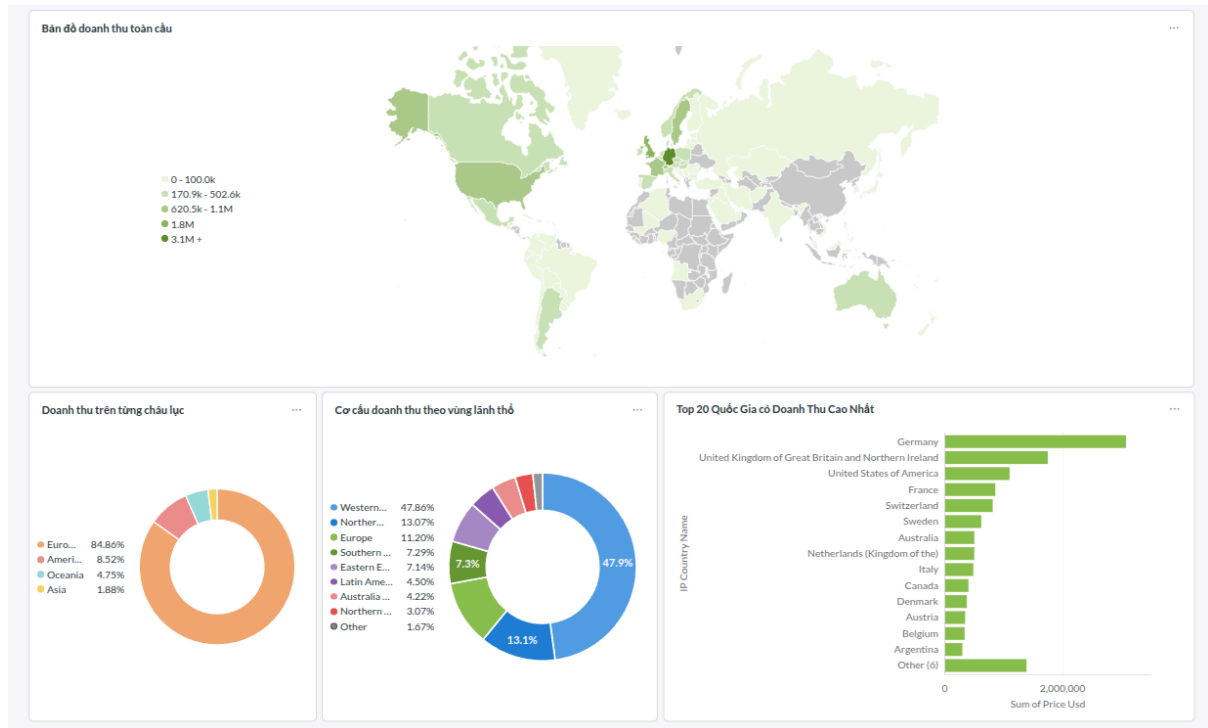


Figure 4.2. Geographical Business Analysis Dashboard (Tab 2)

4.3.1. Spatial Visualization Components

- **Global Revenue Map (Choropleth Map):** Links revenue data with ISO country codes. High-market-share nations are automatically shaded in darker gradients, allowing executives to grasp global market penetration in seconds.
- **Top 15 Revenue-Generating Countries (Row Chart):** A descending bar chart that explicitly identifies the most lucrative core markets keeping the business profitable.
- **Revenue Distribution by Continent (Donut Chart):** Provides a macro-level breakdown of the market structure (Europe, Americas, Asia, etc.).
- **Regional Revenue Breakdown (Treemap):** Displays deeper geographical hierarchies (Sub-regions). Selecting a continent in the Donut Chart dynamically filters this Treemap to show the exact revenue distribution within that specific territory.

- **Actionable Insight:** The geographical data reveals deep market polarization. The enterprise relies heavily on a few key nations (typically high-income countries in Western Europe or North America). This spatial data axis serves as the foundation for the Logistics department to optimize transit warehousing, and for the Marketing department to localize ad copy and language settings effectively.

4.4. Product and Material Performance (Tab 3)

The luxury jewelry sector relies heavily on the synergy between design aesthetics and raw material intrinsic value (gold, silver, diamonds). Tab 3 dissects this product catalog structure.

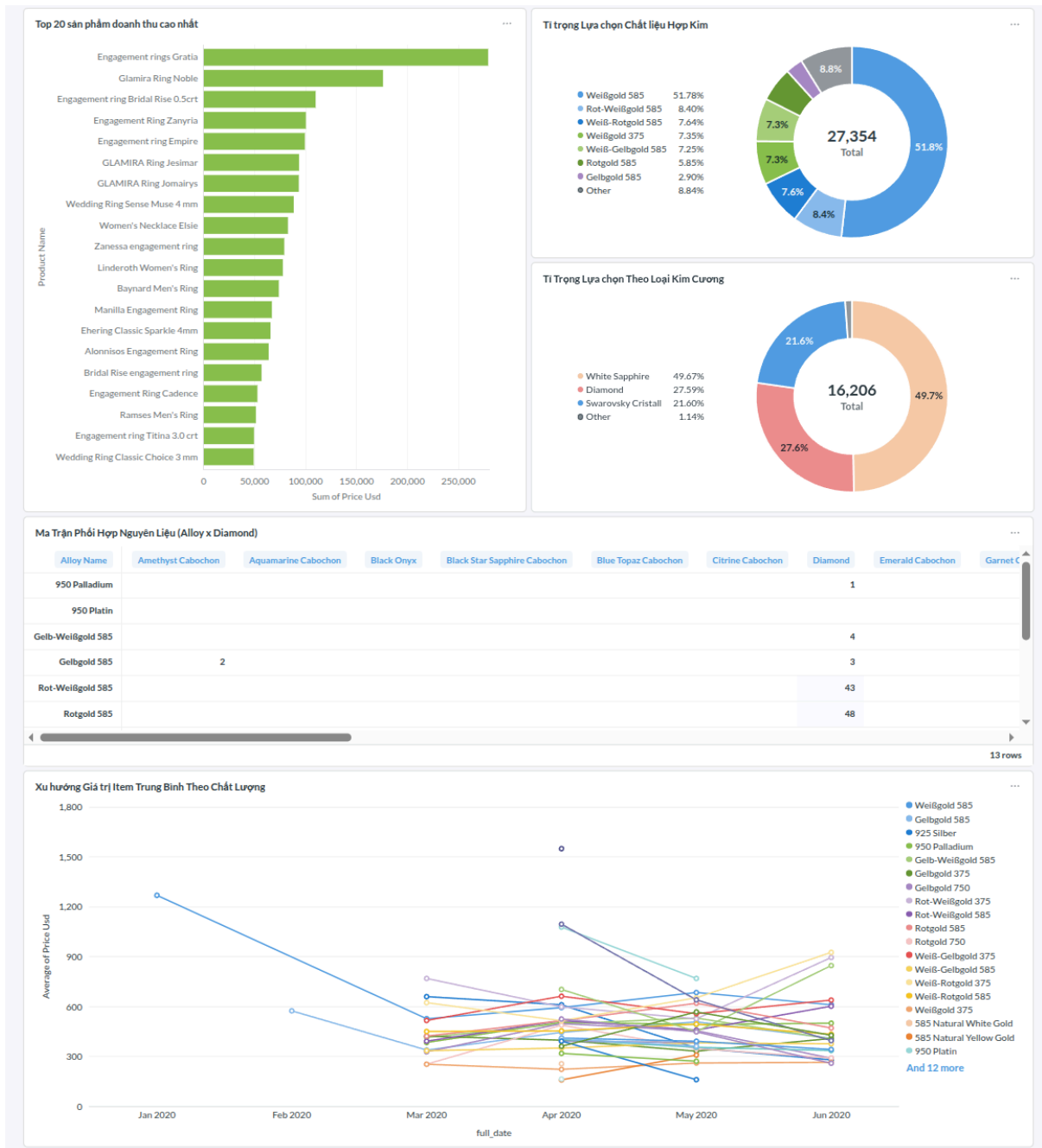


Figure 4.3. Product and Material Performance Dashboard (Tab 3)

4.4.1. Product Portfolio Visualizations

- **Top 20 Highest-Grossing Products (Row Chart):** Identifies the specific SKUs acting as the company's "cash cows."
- **Revenue Share by Alloy Material (Donut Chart):** Analyzes consumer preferences across metal types (Yellow Gold, White Gold, Silver, Platinum).
- **Revenue Share by Diamond Type (Donut Chart):** Quantifies market demand for various gemstone and diamond attachments.

- **Alloy x Diamond Synergy Matrix (Heatmap Table):** A cross-tabulation matrix mapping Alloy names (Rows) against Diamond types (Columns). The volume of items sold triggers the color intensity.
- **Average Item Value Trend by Material (Multi-line Chart):** Tracks the monthly fluctuation in the average selling price (AVG(price_usd)) for different material groups.
- **Actionable Insight:** Consumer trends are starkly visible: Although silver products generate high sales volume, their total revenue contribution cannot match high-end combinations of Gold and Natural Diamonds. The Synergy Matrix informs the design and procurement teams exactly which combinations (e.g., White Gold with Moissanite) are best-sellers, enabling them to focus manufacturing resources and plan clearance sales for poorly performing material pairs.

4.4.2. Actionable Insights and Strategic Recommendations

Through the granular analysis of product and material performance, several critical business insights emerge:

- **Volume vs. Value Discrepancy:** Consumer trends are starkly visible. While lower-tier materials (such as 925 Silver) drive high transaction volumes and initial customer acquisition, the overwhelming majority of the gross revenue is generated by premium material synergies, specifically White Gold (Weissgold 585) paired with Natural Diamonds. This highlights a classic Pareto principle (80/20 rule) within the product catalog, where a small subset of high-end combinations sustains the business's profitability.
- **Inventory and Procurement Optimization:** The Alloy x Diamond Synergy Matrix provides a precise, data-driven blueprint for the supply chain. Procurement teams can dynamically adjust the inventory of raw materials based on these heatmaps. By visually identifying the "hotspots," the business can scale up the acquisition of high-demand combinations while strategically planning clearance sales or phasing out poorly performing material pairs to reduce dead stock.
- **Dynamic Upselling Strategy:** By monitoring the *Average Item Value Trend by Material*, the marketing and UI/UX teams can optimize the website's default interface. Instead of displaying random products, the system's recommendation engine should be hard-coded to prioritize and suggest the highly profitable material combinations identified in the Synergy Matrix, thereby effectively driving up the Average Order Value (AOV).

4.5. Web Traffic and User Behavior Funnel (Tab 4)

The final tab shifts the analytical focus from financial outcomes to digital user experience, utilizing Product Analytics and Web Traffic data.

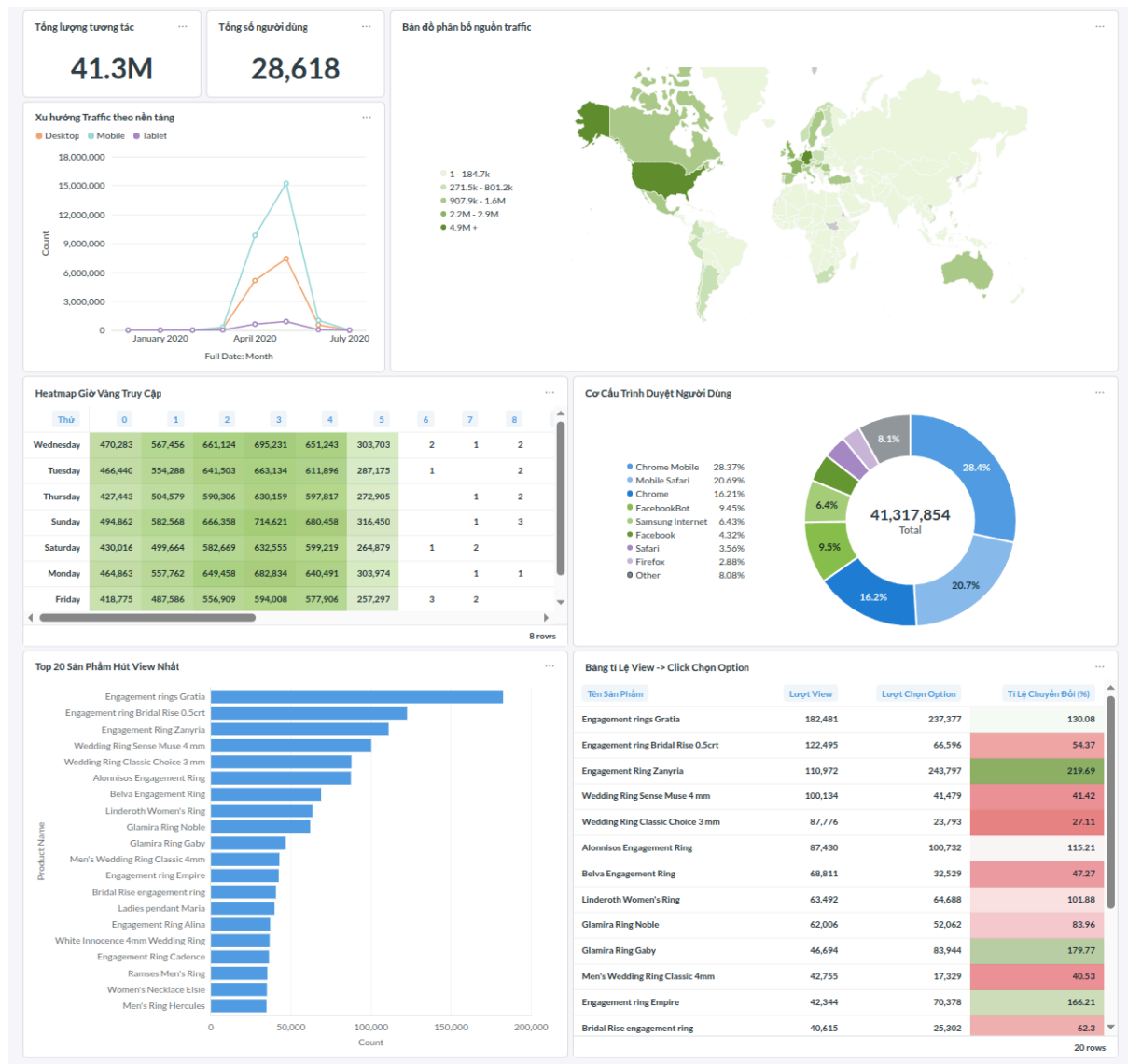


Figure 4.4. Web Traffic and User Behavior Funnel Dashboard (Tab 4)

4.5.1. Traffic Landscape

- **Total Interactions & Unique Users (KPI Cards):** Counts total event_id (network interaction scale) alongside DISTINCT user_id_db (actual customer footprint) to calculate the average session depth per user.
- **Traffic Source Distribution (World Map):** Utilizes IP-Geolocation extracted from ip_country_code to visualize where browsing traffic originates, isolating window-shoppers from actual buyers.

- **Traffic Trends by Platform (Line Chart):** Monitors daily traffic volume segmented by device_type (Mobile, Desktop, Tablet).
- **Peak Access Heatmap (Matrix Table):** Identifies when the web infrastructure faces maximum user load, assisting the DevOps team in server scaling operations.
- **Device & Browser Ecosystem (Donut Chart):** Shows the ratio of Operating Systems (iOS, Android, Windows) and Browsers (Chrome, Safari).

4.5.2. Conversion Performance & Behavioral Funnels

- **Top 20 Most Viewed Products (Row Chart):** Lists the products capturing the highest top-of-funnel attention.
- **View-to-Option Conversion Rate (Table):** A dynamically calculated table comparing Product Views against Option Selections, revealing the actual engagement rate for each design.
- **Behavioral Conversion Funnel (Funnel Chart):** Models the strict customer journey across 4 stages: Product View → Select Option → Proceed to Checkout → Successful Purchase.
- **Actionable Insight:** Cross-referencing these charts exposes significant operational bottlenecks. Some products rank highest in Views but suffer from abysmal Option Selection or Purchase rates. This indicates that while the thumbnail/ad is highly clickable, the actual price point, sizing, or material options deter the buyer. Furthermore, the Funnel Chart highlights the exact stage of maximum drop-off (e.g., Checkout Abandonment). This provides targeted evidence for the UI/UX team to redesign the "Add to Cart" flow or streamline payment gateways to recapture lost revenue.

4.6. BI System Evaluation

Following the deployment of the end-to-end pipeline, from extraction and cleansing to visualization, the BI system demonstrates the following operational results:

4.6.1. Architectural Strengths

- **Exceptional Query Performance:** Leveraging ClickHouse's column-oriented architecture and aggressive deduplication via the LIMIT 1 BY clause at the View layer, the dashboards achieve near-instantaneous load times (sub 0.5 seconds). This performance holds true even during heavy drag-and-drop filtering over massive datasets.
- **Self-Service BI Capability:** The Metabase UI is streamlined and logically partitioned. This empowers non-technical users, such as sales managers and

C-level executives, to independently mine insights through intuitive filters without requiring SQL knowledge or engineering support.

- **Precision Data Cleansing:** The engineering pipeline successfully eradicated timestamp hash collisions via upgraded hashing functions and completely halted fan-out effects during table joins. This guarantees that the aggregated financial figures on the dashboard achieve 100% parity with the raw truth data in the Fact tables.

4.6.2. Technical Limitations and Future Scope

- **Batch Processing Latency (Not Real-time):** The system currently operates on a Batch Processing paradigm governed by scheduled Apache Airflow DAGs. Consequently, dashboard data possesses inherent latency (T+1 or hourly cycles) and cannot reflect real-time user interactions occurring on the website at that very second.
- **Limited Third-Party Integration:** The current data model relies heavily on internal web event logs and fixed transactional data. To deepen strategic insights, the ecosystem requires integration with third-party APIs (such as Google Ads and Facebook Ads). Ingesting marketing spend data would allow the system to automatically compute advanced unit economics, such as Customer Acquisition Cost (CAC) and Customer Lifetime Value (LTV).

CONCLUSION

The Business Intelligence project has successfully architected and deployed a highly scalable, automated Data Pipeline capable of processing over 41 million e-commerce transaction records. By leveraging a modern data engineering stack comprising MongoDB, Apache Spark, Airflow, and ClickHouse, the system effectively bridged the gap between unstructured raw web logs and a rigorous multidimensional data model (Fact Constellation Schema). The resulting interactive dashboards in Metabase successfully addressed the core business objectives, delivering deep, actionable insights into product profitability, revenue seasonality, and geographic customer behavior. Ultimately, the system empowers business stakeholders to transition from intuition-based management to true Data-Driven Decision Making.

Despite these significant achievements, the current system possesses certain limitations. Primarily, the pipeline relies on batch processing (e.g., daily or hourly ETL runs), meaning insights are generated with a slight time delay rather than in true real-time. Additionally, the depth of the customer analysis is currently constrained by the available internal dataset, lacking external market indicators or demographic enrichment.

Looking forward, future developments will focus on integrating message brokers like Apache Kafka to transition the architecture from batch processing to real-time event streaming. Furthermore, integrating Machine Learning algorithms (such as Spark MLlib) will elevate the system from descriptive to predictive analytics. This will enable proactive revenue forecasting and advanced RFM (Recency, Frequency, Monetary) customer clustering, thereby further optimizing targeted marketing campaigns and personalized product recommendations.

REFERENCES

- [1] Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). John Wiley & Sons.
- [2] Inmon, W. H. (2005). *Building the Data Warehouse* (4th ed.). John Wiley & Sons.
- [3] Apache Spark Documentation. (2024). *Spark SQL, DataFrames and Datasets Guide*. Retrieved from <https://spark.apache.org/docs/latest/sql-programming-guide.html>
- [4] ClickHouse Documentation. (2024). *ClickHouse: Fast Open-Source Column-Oriented Database*. Retrieved from <https://clickhouse.com/docs/en/>
- [5] Apache Airflow Documentation. (2024). *Airflow Architecture Overview and Concepts*. Retrieved from <https://airflow.apache.org/docs/>
- [6] MongoDB Documentation. (2024). *MongoDB Spark Connector Guide*. Retrieved from <https://www.mongodb.com/docs/spark-connector/current/>
- [7] Metabase Documentation. (2024). *Business Intelligence, Dashboards, and Data Visualization*. Retrieved from <https://www.metabase.com/docs/latest/>